

Ce este un sistem distribuit ? - e un sistem ale carui componente sunt localizate pe diferite computere legate intr-o retea si care comunica si se coordoneaza prin trimiterea de mesaje unul altuia.

Caracteristicile principale ale acestuia ar fi : - concurenta componentelor, lipsa unui clock global, esecul independent al componentelor

Ce este o baza de date distribuita ? – o baza de date care e stocata in mai multe locatii fizice. Pot fi mai multe calculatoare care se afla in aceeasi locatie sau poate fi imprastiata pe mai multe calculatoare interconectate. Sunt alcatuite din parti slab cuplate care nu impart componente fizice

Ce este o aplicatie distribuita ? – o aplicatie software stocata intr cloud care ruleaza pe mai multe sisteme in acelasi timp, simultan.

Ce este un serviciu web? - o aplicatie modulara care poate fi descrisa, publicata, localizata si invocata pe web. (mai simplu, o componenta independenta, a carui stare si functionalitate nu depinde de intreg sistemul). Ca exemplu si descriere, poate fi un serviciu oferit de un dispozitiv altui dispozitiv sau un server care asteapta request uri si creeaza aplicatii web(API-uri) ce pot fi folosite de acolo doar pentru ce ai tu nevoie. Acesta poate fi descoperit de alte servicii si folosit. De exemplu ca in licenta, poti include in aplicatie un API care sa iti preia informatiile despre hardware-ul telefonului tau si tu doar sa stochezi datele respective la un server.

- Suplimentar : cum utilizam aceste servicii web ? ele sunt invocate folosind mesaje SOAP, care sunt codate in XML si transmise cu ajutorul protocolului HTTP. (schimbul de mesaje e definit in WSDL = Web Service Description Language)
 - o Tipuri de modele :
 - sincron – cand clientul se conecteaza, primeste un raspuns si e de tipul request – response
 - asincron – clientul nu este interesat de raspuns, serviciul preia doar mesajul si trimite raspunsul in momentul cand este gata procesarea

SOA(Service Oriented Arhitecture) este un set de guideline uri pentru a relaliza sisteme software slab cuplate(este legate de arhitectura/design) – SE AXEAZA PE **ORCHESTRARE**(Ce inseamna asta ? - ai un serviciu coordonator, care apeleaza intr-o anumita ordine alte servicii ... mai exsita si **COREGRAFIE**, un serviciu se apeleaza unul pe altul)

- program software
- arhitectura tehnologica
- infrastructura tehnologica

- 3 maniere de implementare a abordarii SOA :

- JBeans
- servicii web (o va matura serverless computing)
- bazata pe componente

* **bazata de componente** : incapsulare in serviciu (bataie de joc referitor la resurse)

* **servicii web** : bazat pe scheme XML si cu probleme grave de securitate

* **servicii REST (SOA)** : simplitate, scalabilitate, reutilizare, pt chestiuni simple

(SOA != REST, REST este un <modul?> din SOA)

- **SWOT SOA** : S :
- fiabilitate
- scalabilitate

W : - incarcare a sistemului (continuu, o aplicatie gen facebook : cand noi dormim, americanii nu dorm)

- costuri initiale mari
- gestiune complexa a serviciilor

SOAP(Simple object access Protocol) – un protocol (set de reguli) care ajuta serviciile web sa comunice intre ele

- nu poate utiliza REST fiindca e protocol
- utilizeaza interfata serviciului(contractul) pentru a expune logica de afaceri
- defineste propriul standard de securitate
- lucreaza doar cu formatul XML

Monolit(o singura unitate)

- vin request-urile printr-un frontend layer
- se trec printr-un load-balancing layer
- si ai servere care hosteaza copii complete ale aplicatiei
- fiecare copie contine toate feature-urile si functiile pe care le poate oferi aplicatia, fie ca le folosesti fie ca nu

ROI – se refera ori la return of investment ori la swarm la docker

- **Swarm** e orchestrator de containere, are grija sa creeze mai multe instante de containere

CONTAINER – e o metoda prin care decuplezi aplicatia, poti sa le compare cu niste masini virtuale, sunt niste sisteme de operare care ruleaza independent de os ul gazda si poti comunica cu ele prin retea

- locul in care instalam aplicatiile, iar aceste containere adauga tot ce trebuie ca aplicatia sa mearga (plugins, dependencies, etc) - se inghesuie totul in container

CI/CD – pipeline prin care trece buildul (de exemplu Jenkins)

- pe partea **CI** se fac teste si aplicatia trece prin diferite environmenturi si automatisme
- **CD** se ocupa de teste end to end si teste de user efectiv si apoi da deploy pe productie

BIG DATA – se refera la un volum foarte mare de date care ar dura maim ult sa fie procesat in mod normal si asa apar diferite fluxuri de procesare cum e map reducer

CONTRACT – se refera la contractual serviciului, un serviciu are un contract de operatii pe care il prezinta la un consumtor

NOR – imbinare intre grid computing si reseaua de internet, pune la dispozitie mai multe servicii si pot fi consummate, aici apar acele tipuri SAAS, PAAS, IAAS

Cu **IaaS**, producatorul manageriaza de la OS in jos, si tu de la OS in sus(mult mai convenabil)

Cu **PaaS**, producatorul manageriaza de la runtime in jos, tu doar aplicatiile si data

Cu **SaaS** Se ocupa provider-ul de tot

FaaS(Function as a Service) : exemple : API Gateway, Function Watchdog, Prometheus, Swarm, Jubernetes, docker

XaaS(Orice ca Serviciu) : Storable, DB, Communication, Network, Monitoring, Testing, HPC, Human

RMI(Java Remote Method Invocation) – registru ce tine evidenta obiectelor , introduce ideea de stub

- stub : o aplicatie n-are nevoie de toate obiectele ca sa lucreze asa ca aducem toate obiectele critice apoi aducem si restul si daca era nevoie de un obiect (de ex in cazul compilarii) se folosea un fel de proxy

(stub) care aducea si obiectul respective

- Programul client va invoca metode din programul server(unde sunt stocate instantele si metodele) – dar pt a face asta va trebui sa promim procesul, ce presupune interogarea unui registru rmiregistry ce ne va spune daca acel obiect este valabil pt client (totodata si clientul foloseste acest registru pentru a gasit referinte la server pentru obiectele remote). Pentru ca clientul poate introduce clase derivate, trebuie sa facem aceste clase vizibile si pentru server, de asta avem legatura prin protocoale HTTP/URL catre client server si invers.

DOCKER – este un set de platforme ca produse de servicii care utilizează virtualizarea la nivel de sistem de operare pentru a livra software în pachete numite containere.

– Exemplu de FaaS(Function as a Service)

JEE BEANS – clasificare si explicatii -

- multinivel, multistrat
- cadru pentru dezvoltare rapida a aplicatiilor
- instalarea implica hard heterogen(sisteme cu mai multe tipuri de procesor/nuclee)

CONSUMATOR – cand un serviciu e consumator, preia rezultatele de la altii si le foloseste.

IoE(Internet of everything) – este un concept care extinde ideea de IoT(internet of things) punand accent pe comunicare masina-la-masina pentru a descrie un system mai complex care implica oameni si procese.

IoT(Internet of things) – este un concept mai larg, include orice tip al unui obiect fizic sau virtual care poate fi facut adresabil si face astfel posibilitatea transmiterii intre masina-om a input ului. In principal este vorba despre lucruri care nu au fost prezente pe internet in trecut, despre automatizare si comunicare (ex. : senzori).

Serviciu middleware - este un program care furnizeaza servicii aplicatiilor software, altele fata de cele care deja sunt puse la dispozitie de sistemul de operare

-face pentru developer mult mai usoara implementarea comunicarii si i/o(UTILIZAREA LOR : le poti controla mult mai usor decat sa te apuci sa rescii sistemul de operare de unul singur)

SWOT SISTEME DISTRIBUITE : Analiza SWOT(Strengths, Weaknesses, Opportunities and Threats)

S:

- costuri reduse(faptul ca poti folosi chestii de la alte firme, fara sa mai iei tu tot hardul, doar folosesti)
- modularitate si flexibilitate(in loc sa proiectezi aplicatii monolit, le poti imparti in module/servicii care pot fi extinse independent)
- fiabilitate si integritate(sistemul functioneaza fara defecte intr-un interval de timp si spatiu dat)
- performanta(poti lucra in paralel, si nu esti limitat de hard-ul de pe o singura masina)

W:

- lipsa cunostintelor despre starea globala(de obicei serviciile sunt stateless)
- lipsa unui timp global(pot aparea intarzieri datorate lipsei de sincronizare a clock-urilor la nivel global)
- nedeterminismul(nu stii mereu cui se trimit pachetele si intotdeauna trebuie verificat)
- comunicatiile(trebuie ca conexiunea sa fie buna pentru a nu scadea viteza sistemului din cauza transmisiei datelor)

- securitatea (trebuie securitate buna ca sa nu se afle date confidentiale trimise prin retea)

- **sisteme k tolerante** : exemplu lift : pt functionare e nevoie de un singur cablu, dar se folosesc 3 pentru siguranta (=> 3 tolerant)

- **arhitectura multinivel-multistrat** : mai multe niveluri care au mai multe straturi, astfel se realizeaza decuplarea functionalitatilor – SOLID

- **injectarea dependentelor** – proxy, apel de constructor, se poate face manual/automat (ca mini definitie, aceasta este un sablon de proiectare software care permite scotarea dependintelor hard-codate si face posibila schimbarea lor, fie in timpul compilarii sau al rularii) – dependintele pot fi introduce fie prin injectie in constructor, apel al metodelor sau prin setarea proprietatilor
 - operatie de scriere, verificare corectitudine, verificare destinatie si sursa corecte => fiecare se modeleaza separat, pe baza de obiecte si sa respecte si SOLID, adica fiecare dintre operatii sa poata fi folosita independent. putem si sa extindem clasa imbox cu aceste functionalitati dar pleaca SOLID instant
 - injectare dependenta inseamna ca avem toate aceste servicii si facem ca email-ul sa depinda de ele.
 - injectia : esentiala in modelarea grafurilor de obiecte, doar ca incarca obiectul deci este problematica intrucat modifica intr-un colt dar strica in alt colt
 - o alternativa buna era fabrica de obiecte (acum 40 de ani)

- **REST(Representational State Transfer)** – este doar un mod fancy de a spune ca acel website/serviciu web presupune metode de create/read/update/delete (adica se trateaza fiecare response al serverului ca un endpoint – exact cum am facut la POS)
 - arhitectura software
 - poate utiliza SOAP pentru mesaje
 - utilizeaza URI pentru a expune logica de afaceri. Poti mapa mai multe operatii pe acelasi URI
 - preia securitatea protocolului de transport utilizat
 - accepta diverse formate(XML, JSON, HTML)

JEE:

- multinivel, multistrat
- cadru pentru dezvoltare rapida a aplicatiilor
- instalarea implica hard heterogen(sisteme cu mai multe tipuri de procesor/nuclee)

- Tipuri de BEAN (intrebare posibila JEE Beans)

- **fara stare** : dependency injection (Java are pointeri si acces la memorie!!nu se ocupa GC de tot)

- **cu stare** : creare, dependency injection, post construct, init method sau ejbCreate

- **orientat pe mesaje** : multe cozi (ca la stare) (vezi mai bine in curs!)

- Spring - dezvoltare usoara de aplicatii de intreprindere, prezint scripturi de automatizare

- poate pentru o clasa vrem sa creem doar o singura instanta pentru intreaga aplicatie, cum putem sa facem aceasta ? Adaugam spring framework, si ea realizeaza un container al aplicatiei (spring application context) si spring se ocupa de crearea si manageriaza instancele(fiecare clasa declara dependintele de care are nevoie), se ocupa cu injectarea dependintelor si stie ca fiecare clasa are tot ce ii trebuie.

- presupune multe API uri pentru conectare la o baza de date (impotriva JDBC urilor,le ascunde si face experienta mult mai placuta la dezvoltarea codului)

- vine cu un framework SPRING MVC , care creeaza aplicatii single page utilizand API uri.

Spring e format din

1) Core Container:

- a) Beans
- b) Core
- c) Context
- d) SpEL(Spring expression Language)

2) AOP

3) Aspects

4) Instrumentation

5) Messaging

6) Test(pentru testare aplicatii)

7) WEB(MVC/REMOTING)

- a)WEB
- b)Web Socket
- c)Servlet
- d)Portlet

8) Data access

- a) Transactions
- b) OXM(Object XML Mapping)
- c) ORM(Object Relational Mapping)
- d) JMS(Java Messaging Service)
- e) JDBC(Java DB conectivity)

1)a)Beans

Spring se foloseste de **Inversion of Control(IoC)** care se mai numeste si dependency injection care e un proces unde obiectele isi defines dependentele(adica celelalte obiecte cu care lucreaza) doar prin argumente la constructor/argumente la o metoda factory(sau proprietatile care sunt setate pe instanta unui obiect dupa ce e construit/returnat de factory. Containerul injecteaza dependentele respective cand creeaza BEAN-ul. Nu mai controleaza bean-ul instantierea si localizarea ci containerul in sine.

Deci un bean e creat, asamblat si manageriat de IoC container.

Ioc se mai poate defini ca procesul in care un obiect doar isi defineste dependentele fara a le crea.

Trebuie doar sa configurezi cum trebuie IoC si nu te mai doare capul.

Acele configuratii sunt defapt alea cu @ de deasupra claselor si ii zici cum sa instantieze, configureze si asambleze obiectele in aplicatie.

Beanurile incep cu litera mica si sunt camelCase

Poti avea Injection de 2 tipuri:

- **constructor based:** class SimpleMoviePlaylist(private val movieFinder: MovieFinder), fara nicio anotare speciala

- **setter-Based:** constructorul e fara argumente(dar poate avea si din alea), iar containerul va apela metode de set, pentru membrii clasei

(la kotlin poti pune doar lateinit var movieFinder: MovieFinder)

1)b) Core - Cuprinde IoC, AOP si altele

1)c)Context

Reprezinta containerele IoC

1)d) SpEL(Spring expression Language) - E un limbaj ce permite interogarea si manipularea obiectelor la runtime.

Functionalitati:

- expresii literale
- boolean si operatori relationali
- regex
- referinte la beanuri
- selectie de colectii

Il folosesti de obicei pentru parsari

2)AOP(Aspect Oriented Programming)

Completeaza OOP-ul si ofera alt mod de a gandi structura unui program. Unitatea fundamentala la OOP e clasa iar la AOP e aspectul. E declarativ.

De exemplu o functie ar putea suna cam asa: "da log la toate apelurile functiilor ale caror nume incep cu set"

E pentru tranzactii, securitate, logging etc

4)Instrumentation

Iti permite sa adaugi cod binar la clasele java existente si compilate ca sa poti monitoriza performanta, diagnostica erorile etc. De obicei se realizeaza prin logging.

5)Messaging

Lucrezi cu servicii care comunica prin mesaje pentru a fi slab cuplate. Iti poti alege ce fel de provider de mesaje vrei(JMS,RabbitMQ,Kafka)

pui dependentele necesare cu Maven/Gradle, si apoi inveti cum se utilizeaza fiecare adnotare si cum trimiti/primesti mesaje.

7)a)MVC

In spring request-urile HTTP sunt prelucrate de un controller(la care ii pui adnotatia @Controller)

@GetMapping spune ca ai de a face cu o cerere tip get

@RequestParam leaga valoarea din query-ul din URL de variabila din program

Dupa dai update la model si intorci rezultatul la view care este browser-ul.

7)b)WebSocket - Lucrezi cu mesaje

Daca se trimite mesaje la /hello, se apeleaza metoda greeting care trimite ce proceseaza din mesaj la /topic/greetings

7)c)Servlets

Servlet e un obiect care primeste request-uri si da raspunsuri pe baza lor

7)d)Portlets

E ca un servlet, doar ca daca faci acelasi request de 100 de ori(dai refresh la pagina) se face o singura procesare, si rezultatul e randat de 100 de ori la fel. Se mai proceseaza doar cand se produc schimbari.La servlet se proceseaza de fiecare data.

8)a)Transactions

O abstractizare consistenta pentru tranzactii.

8)b)OXM

Reprezinta procesul de conversie al unui XML in obiect si invers. Se mai cheama XML Serialization.

8)c)ORM

Conversia datelor intre tipuri de sisteme incompatibile folosind limbaje OOP(Java/Kotlin la Spring)

Ai foarte multa flexibilitate, poti face diverse straturi cu parti care nu depind de spring(slide 11)

Stiva spring:

- Spring MVC
- Service Manager
- DAO
- Hibernate(pentru DB)
- DB

Si totul e protejat folosind securitatea Spring

-Gradle/Maven – o colectie de script-uri, nu reprezinta un framework, acestea sunt utilizate la build pentru a face managementul dependintelor

-CLOUD - esenta acestuia este bazata pe virtualizare (ce reprezinta virtualizarea ? o translare binara atunci cand kernelul este apelat --- sau --- abstractizarea sistemului de operare, aplicatiilor si zonelor de

pastrare a datelor sau a retelei de comunicatii fata de suportul hardware si software care permite acest process)

- **Virtualizare completa** : simulare completa a unui hardware sau software

- **pe soft** (translate binara) si emuleaza hardul folosind instructiuni soft compatibile

- **pe hard** : se bazeaza pe suportul nativ de hardware, se pot executa direct pe processor

- **Arhitectura Cloud**

- n level n tier

- echilibrarea incarcarii

- multe cozi

- **Nivel de prezentare Cloud**

- pagini web cu servicii disponibile

- infrastructura elastica

- multiplicare verticala : se poate muta pe alta masina

- multiplicare orizontala : se muta si pe alte masini si se distribuie resursele pt eliminarea incarcarii (trebuie facuta jurnalizare)

- **Internet Tehnologies pentru CLOUD**

1) SOA

2) Web 2.0

3) Web Services

4) **Mashups**: o pagina/aplicatie web care foloseste continut din mai multe surse pentru a crea un nou serviciu afisat intr-o singura pagina web

Caracteristici de baza CLOUD :

- auto-service la cerere

- acces oriunde la retea

- grupare resurse independente

TIPURI DE CLOUD — - public external(public clouds)

- private internal(enterprise clouds)

- **Automation**(nu e specificata in curs, doar i-am pus eu nume)

1) Automatic Computing: caracteristicile de manageriere propriie a unor resurse distribuite de computing, adaptandu-se la schimbari imprevizibile

ascunzand complexitatea intrinseca operatorilor si userilor

2) Data Center Automation: proces prin care rutinele de workflow si procesele unui data center sunt executate si manageriate fara interventia umana

- **Distributed Computing**

1) Utility Computing: un model de aprovizionare cu servicii in care un provider face resursele de computing si infrastructura disponibila clientului

dupa necesitati si il taxeaza in functie de consum.

2) Grid Computing: folosirea la scara larga a resurselor computerelor pentru a atinge un tel comun. Nodurile grid-ului executa o aplicatie/task diferit.

Tipuri de scalari:

- **scale-up/vertical Scaling**

- cererile vin printr-un front-end, sunt preluate de un Load Balancer si apoi serverul hosteaza o copie completa a aplicatiei

- Scalare prin cresterea resurselor(CPU, RAM) a fiecarui host

- **scale-out/Horizontal Scaling**

- scalare prin adaugarea mai multor servere

Principiile SOLID la microservicii :

S:

- un microserviciu ar trebui sa implementeze doar o functie de business

- poti sparge microserviciul coarse-grained in microservicii fine si sa le bagi un orchestrator

O:

- un microserviciu nu ar trebui sa fie niciodata modificat ca sa ofere functionalitati ocazionale sau de exceptie

- in loc sa ai un serviciu ce include cazurile exceptionale, faci un serviciu pentru ele si apelezi serviciul initial

L:

- o versiune noua a unui microserviciu ar trebui intotdeauna sa poata inlocui o versiune veche fara sa strice nimic

I:

- un microserviciu n-ar trebui sa expuna metode care nu-s direct asociate

- daca folosesti doar 30% din microserviciu, atunci ala nu e microserviciu

D:

- un microserviciu n-ar trebui sa apeleze direct alt microserviciu

- ar trebui sa foloseasca un un Service Discovery(gen Docker) sau sa lase aplicatia sa determine la runtime ce ce microserviciu sa invoce

Principii de scalare a microserviciilor

Horizontal Decomposition(X axis)

- scalarea prin adaugarea mai multor instante ale microserviciului in spatele unui load balancer

- adica adaugi mai multe masini

Functional Decomposition(Vertical, Y Axis)

- imparte aplicatia in mai multe servicii diferite

Data Partitioning(Z Axis)

- rulezi mai multe copii identice, dar fiecare se ocupa doar de un subset de date

- poti sa dirijezi spre fiecare serviciu in functie de niste criterii stabilite\

Poti avea multe dependente intre microservicii si pentru o singura cerere sa pierzi mult timp

O solutie ar fi sa implementezi mecanisme de caching:

- vin cererile
- modulul lb(load balancing) transfera cererea
- serviciul verifica daca are in cache cererea respectiva
- daca da, o returneaza, daca nu, face calculele necesare

Exista:

- cache incapsulat(e bagat in serviciu)
- cache comun(e partajat intre mai multe instante ale microserviciului)
- atasament(sidecar) ai un serviciu de cache cu care comunica microserviciul(asigura cuplare slaba si elimina redundantele)
- inverse proxy in care atasezi cache-ul la lbs

Descoperire la client: te uiti intr-un service registry primesti o adresa si te conectezi la microserviciu

Descoperire la server: te conectezi la un server lb/API Gateway care se uita in service registry si-ti da forward.

Poti seta microserviciile sa se autoinregistreze sau sa se delogheze automat sau sa folosesti tool-uri externe

Modele de migrare al aplicatiilor in cloud:

- rehost, iei de la tine si pui in cloud
- refactor, modifichi aplicatia ca sa se potriveasca mai bine pe cloud
- replatform, nu faci mari modificari dar profiti de avantajele oferite
- rebuild, scrii aplicatia de la 0
- replace, retragi aplicatia si o inlocuiesti cu o noua aplicatie cloud nativa

#####

Suplimentare :

- REST si SOAP pot fi considerate ambele implementari ale SOA
- HTTP are Securitate slaba
- Arhitectura in dezvoltarea aplicatiilor trebuie sa fie multilayer/tier si trebuie stabilita tehnologia folosita
- **Infrastructura tehnologica** : infrastructura hardware prin care sunt instalate aplicatiile (nas, ups, routere, aere conditionate, sisteme de operare, agenti, baze de date, sistem de gestiune a cozilor de mesaje, adaptoare, middleware(???), solutii de securitate, etc)
- **relatia afaceri** : abordare preemptiva! daca dezvoltam aplicatia pentru 100 de useri dar vor veni 10000, ce facem ? reprojectam si reimplementam? not a good idea!
- **relatia intre app si mediu de utilizare** : interoperabilitate stransa. cine raspunde de securitate ? tarile civilizate au standarde pe subiect
- **specificul proiectului** : rezulta din maniera de proiectare proiectarea e influentata si de arhitectura ei exterioara citire standard apoi alte vrajeli !
- **model de proiectare** : design patterns : solutie validata empiric pentru specificul unei probleme
- **standard de proiectare** : influentata mult de ideea si principiul de afaceri se pot adapta si la nivel particular suporta imbunatatiri continue standard proiectare != standard industrial
- **calcul orientat pe servicii** : - platforme sau software pt calcul distribuit
 - crearea entitati logice cu grad mare de izolare care pot fi utilizare si reutilizate intre ele pentru a servi proiectarii unei arhitecturi
 - serviciu SOAP != serviciu din SO
 - un IT-ist foarte bun intelege problemele de business (partea economica!)
- **inventarul serviciilor** : ca o carte de telefonie : colectie logica, care poate reprezenta o intreprindere sau o componenta de afaceri intr o intreprindere
- **agnostic** : serviciu total independent
- **RPC(remote procedure call)** apelezi o procedura de pe alt calculator(de obicei printr-o retea) ca si cum ar exista local
- **BPM(Business process modeling)** e activitatea de reprezentare a proceselor unei intreprinderi astfel incat procesul curent sa poata fi analizatimbunatatit si automatizat.
- EAI(Enterprise application integration)** reprezinta folosirea soft-ului si a principiilor arhitecturale ale sistemelor de calatoare pentru a integra un set de aplicatii la nivel de intreprindere

Terminologie SOA :

Design Characteristic:

- o proprietate a unui program software sau **Tehnology Architecture**(TA) care rezulta din modul in care a fost proiectat

- poate fi orice calitate concreta, cum ar fi faptul ca programul e alcatuit din componente, ca functiile au granularitate fina sau aspra

Design Principle:

- practica acceptata de industrie care are un anumit scop
- paradigma SOA e compusa din principii de design care sunt aplicate impreuna pt a atinge obiectivele orientarii pe serviciu

Design Pattern:

- o solutie demonstrata la o problema de design, documentata intr-un format consistent

Design Standard:

- conventii de design particularizate individual de organizatie ca sa livreze solutii in concordanta cu telurile de business specifice organizatiei

Service Orientation:

- paradigma de design pentru crearea unitatilor solutiei logice care sunt proiectate individual ca sa fie utilizate repetat si colectiv in

scopul realizarii unor obiective

- unitatile se numesc SERVICII

Serviciu:

- e o unitate logica asupra careia s-au aplicat intr-o masura semnificativa principiile proiectarii service-oriented

Service Capability:

- fiecare serviciu are propriul context functional distinct si e compus dintr-un set de functii numite capabilitati legate de context

- cercelele alea din desenul serviciului

- nu influenteaza cum e implementat serviciul

Service Consumer:

- cand un program invoca si interactioneaza cu un serviciu se numeste service consumer
- termenul se refera la rolul temporar luat la runtime de un program atunci cand e angajat cu un serviciu pentru schimb de date

Service Provider:

- serviciul invocat de consumer

Service Composition:

- agregare de servicii compuse colectiv pentru a automatiza un anumit task/proces de business

- ca sa avem compozitie, e nevoie minim de 2 servicii participante si un initiator al compozitiei. Altfel e doar schimb

Service Inventory:

- colectie de servicii complementare, standardizate, independente si guvernate, aflata in limite care reprezinta o afacere sau un segment al afacerii

- cand ai mai multe inventare termenul devine domain service inventory

Principiile service oriented:**1)Standardized Service Contract**

Serviciile din acelasi inventory sunt in conformitate cu aceleasi standarde de design

2)Service Loose Coupling

Contractele serviciilor impun cerinte de cuplare scazute pentru consumator si sunt la randul lor decuplate de mediul inconjurator

3)Service Abstraction

Contractele serviciilor contin doar informatii esentiale si informatiile despre servicii sunt limitate doar la ce e publicat in contract

4)Service Reusability

Serviciile contin si exprima logica agnostica(nu conteaza contextul in care sunt chemate) si pot fi considerate resurse enterprise reutilizabile

5)Service Autonomy

Serviciile exercita un nivel ridicat de control asupra mediului de executie underlying la runtime

6)Service Statelessness

Serviciile minimizeaza consumul de resurse prin amanarea managementului informatiilor despre stare cand e necesar

7)Service Discoverability:

Serviciile sunt suplimentate cu date meta de comunicatie prin care pot fi descoperite si interpretate eficient(cum e docker, ai un registru si le iei de-acolo)

8)Service Composability

Serviciile sunt participanti efectivi la compozitie, indiferent de marimea si complexitatea compozitiei

Caracteristici de baza SOA:

a)Business-driven:

- TA e aliniata cu arhitectura curenta de business. Contextul e apoi mentinut constant asa incat TA sa evolueze in tandem cu businessul de-a lungul timpului.

- TA se tot indeparteaza de Business pana in punctul in care trebuie gandita de la 0 si de aia trebuie sa le tii aproape

b)Furnizor neutru:

- Modelul arhitectural nu e bazat strict pe o platforma furnizor, poti combina mai multe tehnologii, sa le inlocuiesti pe unele ca sa maximizezi realizarea

necitatilor afacerii in mod constant

- ai mai multa libertate la implementare

c)Enterprise-centric:

- Scopul arhitecturii reprezinta un segment important al intreprinderii, permitand reutilizarea si compozitia serviciilor, permitand solutii orientate serviciu in

favoarea celor traditionale

- nu iti imparti serviciile pentru anumite lucruri, le pui la dispozitie pentru toti din firma

d)Composition-centric:

Arhitectura suporta mecanici de agregare repetata a serviciilor, permitand acomodarea schimbarilor constante prin asamblare agila a compozitiilor de servicii

SOA Design Patterns:

Inventory Boundary(identifica granitele inventarului)

1)Enterprise Inventory

- Servicii pentru multiple solutii pot fi proiectate pentru livrare intr-un inventory arhitectural standardizat, la nivel de intreprindere unde pot fi recompose repetat si liber

- recomandat pentru organizatii mici/medii

2)Domain Inventory

- serviciile pot fi grupate in inventare de servicii specifice domeniului, fiecare putand fi guvernat, standardizat si detinut independent
- la intreprinderile mari nu mai merge 1) fiindca ar dura prea mult analiza

Inventory Structure(structura high-level si complexitatea generala a unui inventory e data si influenteaza nevoile pe care arhitectura trb sa le indeplineasca)

3)Service Normalization

- inventarul trebuie proiectat cu accent pe alinierea limitelor serviciilor
- serviciile fiind normalizate se elimina riscul ca 2 servicii sa aiba limite functionale suprapuse

4)Logic Centralization

- Accesul la functionalitatea reutilizabila e limitata la serviciile agnostice oficiale
- ca sa nu ai functionalitate redundanta

5)Service Layers

- inventar structurat in 2 sau mai multe straturi logice, fiecare fiind responsabila pt a abstractiza logica pentru un tip functional comun
- minim 2 layere, unul cu logica agnostica(pentru mai multe scopuri), celalalt cu logica non-agnostica(pt un singur scop)

Inventory Standardization(Standardele se stabilesc pt a asigura o baza comuna pentru interoperabilitatea intre servicii)

6)Canonical Protocol

- arhitectura stabileste o singura tehnologie de comunicare ca mediu primar in care serviciile pot interactiona
- doar la comunicarea intre servicii, nu si in interiorul unui serviciu

7)Canonical Schema

- modelele de date pentru seturi de informatii comune sunt standardizate de-a lungul contractelor in limitele inventarului
- schema e ce treci tu de exemplu intr-un fisier xsd(XML schema definition) ca sa ai acelasi format de date intre servicii

Logical Inventory Layer(ai logica utilitara, agnostica si non-agnostica)

8)Utility Abstraction

- un nivel de servicii dedicat procesarilor utilitare, ce ofera servicii utilitare pentru a fi folosite de alte servicii din inventar
- nu e logica de business, ci doar logica utilitara(de exemplu biblioteca matematica)

9)Entity Abstraction

- un nivel de servicii business agnostice poate fi stabilit
- dedicat serviciilor ale caror context functional se bazeaza pe entitati existente de business
- separi serviciile agnostice de cele non-agnostice
- o faci doar daca nu se schimba des specificatiile

10)Process Abstraction

- un nivel parinte de business dedicat
- stabilit pentru a suporta administrarea independenta si a face serviciile task(non-agnostice) potentiale resurse enterprise

Inventory Centralization

11)Process Centralization

- logica ce reprezinta numeroase procese de business poate fi livrata si administrata intr-o locatie centrala
- ai o platforma care orchestreaza toata logica de business

12)Schema Centralization

- selecteaza scheme care exista ca parti fizice separate ale unui contract si sunt partajate intre mai multe contracte
- elimina schemele redundante
- ai WSDL(Web Service Description Language) care e validat de scheme

13)Policy Centralization

- Politici globale sau de domeniu pot fi izolate si aplicate mai multor servicii
- elimini redundanta

- ariile adresate de politici: securitate, conditii de transfer, serie de proprietati QoS(Quality of Service)

14)Rules Centralization

- Stocarea si administrarea regulilor de business se afla intr-o extensie arhitecturala dedicata de unde pot fi accesate si mentinute

- Poti crea un serviciu central pentru a se ocupa de crearea, modificarea, aplicarea si primirea regulilor de business

Inventory Implementation

15)Dual Protocols

- arhitectura inventarului e proiectata sa suporte servicii bazate pe protocolul principal si secundar

- un singur protocol poate sa nu fie capabil sa acomodeze toate necesitatile serviciului, si de aia mai introduci unul

- trebuie folosita in moderare

16)Canonical Resources

- Infrastructura si arhitectura pot fi echipate cu resurse comune si extensii care pot fi utilizate repetat de diferite servicii

- ex de resurse: DB, directoare, extensii de securitate

17)State Repository

- date legate de stare pot fi scrise temporar si apoi sa fie luate dintr-un repo dedicat

- reduci consumul de memorie asociat cu retinerea informatiilor de stare

18)Stateful Services

- Datele legate de stare sunt organizate si stocate de servicii stateful utilitare

- starea e impartita la mai multe servicii utilitare fiind mai usor de organizat

19)Service Grid

- Datele de stare sunt intarziate catre o colectie de servicii stateful care formeaza un grid ce ofera scalabilitate, toleranta la esec

- prin replicare a memoriei si redundanta suportand infrastructura

- eviti riscul de a pierde date in partile super folosite

20)Inventory Endpoint

- capabilitatile relevante sunt abstractizate intr-un serviciu de capat care se comporta ca un punct de intrare oficial in inventar si e

- dedicat unui set de consumatori externi

- oferi doar anumite functionalitati din inventar, nu acces la tot inventarul

- nu fac parte din inventar si nu participa la compozitii

21)Cross-Domain Utility Layer

- un nivel utilitar comun poate fi stabilit, intinzandu-se intre 2 sau mai multe inventare de domeniu

- ai redundanta inutila la logica utilitara

- poti imparti in logica utilitara specifica domeniului, si logica utilitara inter-domeniu

Inventory Governance

22)Canonical Expression

- contracte standard folosind conventii de numire

- contractele pot exprima capabilitati similare in diferite moduri, si nu vrei asta

23)Metadata Centralization

- metadatele serviciilor pot fi publicate central intr-un registru ce ofera un mijloc formal de a inregistra si descoperi serviciile(docker)

24)Canonical Versioning

- regulile de versionare a contractelor si expresia informatiilor de versionare sunt standardizate in limitele inventarului

- poti avea versiuni de contracte diferite in inventar si asta cauzeaza probleme

Service Identification

25)Functional Decomposition

- poti sparge problema intr-un set de probleme mai mici si solutia la fel

- eviti aplicatiile monolit

26)Service Encapsulation

- Logica solutiei e incapsulata de un serviciu care e pozitionat ca o resursa enterprise capabila de a functiona dincolo de limitele pentru

care a fost conceput initial

- trebuie sa proiectezi pentru mai mult de un singur mediu pentru a nu fi limitat

Service Definition

27)Agnostic Context

- Izoleaza logica care nu e specifica unui singur motiv, in servicii separate cu logica agnostica

28)Non-Agnostic Context

- Logica non-agnostica potrivita pentru incapsularea serviciilor poate fi localizata in serviciul care e gazduit ca membru oficial al unui inventar

- pot fi controllere/initiatori ai compozitiilor

29)Agnostic Capability:

- Serviciul agnostic e partitionat intr-un set de capabilitati bine definite care adreseaza preocupari comune care nu-s specifice niciunei probleme.

Prin analiza se rafineaza si mai mult

Service Implementation

30)Service Facade

- o componenta fatada e folosita pentru a abstractiza o parte din serviciile cu potential negativ de cuplare

- ajuta serviciul principal prin niste procesari suplimentare.

31)Redundant implementation

- Serviciile reutilizabile pot fi livrate prin implementari redundante sau cu suport la erori

- eviti single point of failure

32)Service Data Replication

- serviciile pot avea propriile DB cu replicare pentru datele partajate

33)Partial State Deferral

- chiar daca serviciile trebuie sa ramana stateful, o parte din datele de stare pot fi intarziate temporar

- tii doar o parte din stare in memorie

34)Partial Validation

- un consumer poate fi proiectat sa valideze doar partea relevanta a datelor

35)UI Mediator

- Stabileste un mediator care e responsabil doar cu asigurarea interactiunii la timp si feedback de la UI si prezentare

Service Security

36)Exception Shielding

- exceptia unsafe e inlocuita cu una care e safe din proiectare si data la consumer

37)Message Screeing

- serviciul e echipat sau suplimentat cu rutine de ecranare care pp ca orice input e daunator pana se dovedeste contrariul

38)Trusted subsystem

- serviciul e proiectat pentru a folosi credentiale proprii pentru autentificare si autorizare legata de resursele backend in numele consumerului

39)Service Perimeter Guard

- un serviciu intermediat e stabilit ca perimetru al retelei locale ca un punct de securitate pentru orice consumer extern care trb sa interactioneze
cu serviciile interne

Service Contract Design

40)Decoupled Contract

- contractul serviciului e fizic decuplat de implementare si pot varia independent

41)Contract Centralization

- accesul la serviciu e limitat de contract, fortand consumerii sa evite cuplarea cu implementarea

42)Contract Denormalization

- Contractele pot include o cantitate masurata de denormalizare, permitand mai multor capabilitati sa exprime redundant functionalitati de baza in
diverse moduri pentru diferite tipuri de consumatori

43)Concurrent Contracts

- Mai multe contracte pentru acelasi serviciu, fiecare pentru un tip de consumer

44)Validation Abstraction

- Validarea granulara si regulile pot fi abstractizate de contract, reducand granularitatea serviciului si crescand longevitatea contractului
- contractele cu multe constrangeri devin invalide cand se petrece o schimbare

Legacy Encapsulation

45)Legacy Wrapper

- un serviciu wrapper non-standard poate fi inlocuit sau impachetat intr-un serviciu standard care incapsuleaza si poate elimina detaliile tehnice legacy din contract
- impachetezi sistemul legacy intr-un serviciu

46)Multi-Channel Endpoint

- un serviciu intermediar e proiectat sa incapsuleze sistemele specifice canal si sa expuna un singur contract standard pt consumatori specifici canal multipli
- se refera cand proiectezi de exemplu pentru pc/telefon ca ai mai multe feluri de sist legacy, si vrei sa ai un singur serviciu care comunica cu toate device-urile

47)File Gateway

- logica de procesare bidirectionala intre sistemele legacy si servicii ca sa poata comunica, fiindca nu se pot invoca unele pe altele

Service Governance

48)Compatible Change

- unele schimbari ale contractului pot fi compatibibile inapoi, evitand impactele negative ale consumatorilor

49)Version Identification

- informatiile de versionare referitoare la schimbari compatibile si incompatibile pot fi exprimate ca parte a contractului, pentru motive de comunicare si intarire
- minor version nu afecteaza userii, major da

50)Termination Notification

- Contractele pot fi proiectate sa exprime informatii de terminare pentru consumerii umani si programati
- "suportul pentru aceasta versiune se va termina incepand cu data..."

51)Service Refactoring

- Contractul e pastrat pentru a mentine dependentele cu consumerul, dar logica serviciilor underlying e refacuta
- poti avea serviciul la v2 si contractul la v1

52)Service Decomposition

- un serviciu se poate descompune in 2 sau mai multe servicii mai fine

53)Proxy Capability

- contractul original e pastrat, chiar daca logica de dedesubt e separata, prin transformarea definitiei capabilitatilor intr-un proxy
- adica stie contractul ce sa apeleze din serviciu

54)Decomposed Capability

- serviciile supuse riscului de a fi supuse descompunerilor ulterioare pot fi echipate cu o serie de capabilitati granulare care pot facilita mai usor descompunerea

55)Distributed capability

- partea underlying a serviciului e distribuita, permitand implicit ca o capabilitate cu necesitati unice de procesare sa fie separata, in acelasi timp continuand sa fie reprezentata de acelasi contract
- logica de capabilitate e in alta parte de obicei

Capability Composition

56)Capability Composition

- Cand logica necesara e in afara, capabilitatea din serviciu e proiectata sa compuna una sau mai multe capabilitati in alte servicii

57)Capability Recomposition

- Capabilitatile serviciilor agnostice pot fi proiectate pentru a fi invocate repetat pentru a suporta mai multe compozitii care rezolva multiple probleme
- folosirea unui serv agnostic ca sa rezolvi o singura problema, nu creste gradul de reutilizare

Service Messaging

58)Service Messaging

- serviciile pot fi proiectate sa interactioneze prin mesaje care reduce conexiunile persistente si necesitatile de cuplare

59)Messaging Metadata

- continutul mesajelor poate fi suplimentat cu metadata specifice actiunii care sunt interpretate si procesate separat la runtime

60)Service Agent

- service agents pot fi proiectati sa raspunda automat unor conditii predefinite fara invocarea printr-un contract
- event driven

61)Intermediate Routing

- caile mesajelor pot fi determinate dinamic prin logica intermediara de rutare

62)State messaging

- in loc sa retii in memorie, stochezi temporar in mesaj

63)Sevice Callback

- un serviciu poate necesita ca consumerii sa comunice asincron cu el si sa ofere o adresa callback unde serviciul sa poata trimite raspuns la mesaje

64)Service Instance Routing

- serviciile ofera un id de instanta impreuna cu informatii de destinatie intr-un format standard care fereste consumatorul ce a avea sa recurga la logica custom
- ca sa mentii starea in caz ca ai nevoie de un schimb de mesaje

65)Async Queuing

- un serviciu poate schimba mesaje cu consumerii printr-un buffer intermediar, permitand serviciilor sa proceseze mesajele independent prin ramanerea decuplat temporar

66)Reliable Messaging

- un mecanism de incredere intermediar e introdus in arhitectura inventarului, asigurand ca livrarea mesajelor e garantata
- pentru medii unreliable

67)Event Driven Messaging

- consumerul se stabileste ca subscriber al serviciului. Serviciul notifica automat eventurile relevante la toti subscriberii

Composition Implementation

68)Agnostic Sub-Controller

- logica reutilizabila agnostica inter-serviciu e abstractizata sau e facuta disponibila printr-o capabilitate agnostica de sub-controller, permitand subsetului sa fie recompus independent

69)Composition Autonomy

- Toti participantii la compozitie pot fi izolati sa maximizeze autonomia compozitiei ca intreg

70)Atomic Service Transaction

- Actiunile de runtime pot fi impachetate intr-o tranzitie cu capacitati de rollback care reseteaza toate actiunile si schimbarile daca parintele nu poate fi completat cu succes

71)Compenstating Service Transaction

- Rutine de compensare sunt introduse, permitand exceptiilor runtime sa fie rezervate cu lock redus pe resurse si memorie redusa

- poti consuma prea multe resurse folosind 70)

Service Interaction Security

72)Data Confidentiality

- mesaje sunt criptate

73)Data Origin Authentication

- un mesaj poate fi semnat digital si se poate verifica originea si daca nu a fost alterat pe parcurs

74)Direct Authentication

- trebuie ca utilizatorii sa dea credentiale in mesaj ca sa foloseasca serviciul

75)Brokered Authentication

- Un broker de autentificare ca un depozit centralizat de identitati, isi asuma responsabilitatea pentru autentificarea userului si emiterea unui token pe care consumerul il poate folosi sa acceseze serviciul

Transformation Patterns

76)Data Model Transformation

- O tehnologie de transformare a datelor poate fi incorporata sa converteasca datele intre scheme diferite

77)Data Format Transformation

- Logica de transformare intermediara a formatului datelor trebuie introdusa ca sa traduci dinamic dintr-un limbaj in altul

78)Protocol Bridging

- Logica de bridge e introdusa pentru a permite comunicarea intre diferite protocoale prin convertirea dinamica a unui protocol in altul

Enterprise service bus(ESB)implementeaza un sistem de comunicare intre aplicatii soft in SOA

Dirijarea dinamica se refera la trimiterea spre anumite servicii in functie de continutul mesajului.

Filtrul e fix filtrarea mesajelor din Design Patterns

-CLOUD

- Paravirtualizare

- XEN (???)

- peste sistemul gazda, nu se instaleaza un sistem de operare oaspete full, ci modificat astfel incat sa se potriveasca cu hardware ul; aceasta se face cu niste drivere instalate in spate de masina virtuala

- elimina incarcarea excesiva

- Virtualizare la nivel de sistem de operare

- Oracle Solaris

- Docker (??? pt tema)

- Terminologie si concepte Cloud

- Hiperconvergenta ???

- Centru de calcul cu patru straturi

- Platforma hip

Servicii Amazon

- Elastic Compute Cloud
- Simple Storage Service
- Simple Queue Service
- SimpleDB
- CloudFront

Windows Azure

- Live Services
- .NET Services
- SQL Services
- SharePoint Services
- Microsoft Dynamics CRM Services

Design Patterns Microservicii

a)Decomposition Patterns

1)Decompose by Business Capability

- poti descompune dupa capabilitatile de business(ceva ce un business face ca sa produca o valoare)

2)Decompose by subdomain

- ai clase care nu-s usor de descompus si au elemente comune(Order Management, Order Taking, Order Delivery, etc)

- folosesti subdomenii si bounded contexts pentru a rezolva problema
- fiecare subdomeniu va avea un model si un scop care se va numi bounded context
- fiecare microserviciu va fi livrat cu contextul bounded

3)Strangler

- tot faci call-uri de la microserviciu la monolit si invers pentru fiecare domeniu.
- un serviciu poate fi impartit in diferite domenii si hostat ca servicii separate.
- ideea e sa o faci un domeniu la un moment dat.
- asta creaza doua aplicatii separate care lucreaza cot la cot in acelasi spatiu URI.
- noua aplicatie inlocuieste treptat vechiul monolit si la un moment dat poti inchide monolitul definitiv

b)Integration Patterns

4)API Gateway

- singurul punct de intrare pentru fiecare apel de microserviciu
- poate functiona ca un proxy si sa te directioneze la microserviciul potrivit, abstractizand detaliile producatorului
- poate grupa un request la mai multe servicii si sa trimiti un raspuns grupat
- poti avea mai multe API-uri pentru diferiti clienti
- Poti converti un protocol in alt protocol(actionezi pe post de protocol adapter)
- poate lua responsabilitatea de autorizare/autentificare de la microserviciu

5) AGGREGATOR la microservicii - Aggregator

- spune cum putem grupa datele de la diferite servicii si sa trimitem un raspuns final consumerului
- se poate face in 2 moduri: un microserviciu composite care face apeluri la microserviciile necesare, consolideaza
datele si le transforma inainte de a le trimite inapoi(adica orchestrarea de la SOA, eventual un adapter)
- API Gateway poate face request-uri la mai multe microservicii si sa uneasca datele inainte de a le trimite la consumer
- daca ai logica de business mergi pe prima varianta

6)Client-Side UI Composition

- cu microserviciile, UI-ul trebuie sa fie proiectat ca un schelet cu mai multe sectiuni per ecran/pagina

- fiecare sectiune face apel la un microserviciu backend ca sa-si ia datele.

- popular la Single Page Applications

c)Database Patterns

7)Database per Service

- ai o db per microserviciu

- privata si accesibila doar din microserviciu

8)Shared Database per Service

- aplicata doar la brownfield(aplicatii in care ai si monolit)

- ai 2-3 microservicii care impart o db

9)Command Query Responsibility Segregation(CQRS)

- cand fiecare e cu db lui, query-urile care necesita date de la mai multe microservicii nu pot fi facute

- solutia este sa imparti aplicatia in 2 parti: comanda si query.

- comanda se ocupa de CUD(Create Update Delete)

- partea de query face query-uri prin view-uri materializate

- folosit in combinatie cu pattern-ul de event sourcing unde creezi evenuri pt fiecare schimbare a datelor.

- view-urile materializate sunt tinute la curent prin abonarea la streamul de evenuri.

- sau faci o replica read-only a bazelor de date unite

10)Saga

- nu poti folosi tranzactii ACID daca-s in DB diferite

- un saga e un porces high-level de business care e fosrmat din mai multe sub-request-uri in care fiecare

- face update intr-un singur serviciu.

- fiecare request are un request compensatoriu care e executat cand request-ul esueaza
- ai 2 moduri: coregrafie cand nu ai coordonator, fiecare serviciu produce si asculta eventurile altui serviciu si decide

daca o actiune trebuie luata sau nu

- orchestrare cand un obiect ia responsabilitatea pentru decizie si dirijeaza logica de business

11)API Composer

- cum implementezi query-uri care au nevoie de join de la mai multe microservicii
- faci un serviciu care face query la fiecare serviciu si da join la rezultate

12)Domain Event

- cum poate emite un serviciu evenimente cand isi da update
- organizeaza logic de business ca colectii care emit eventuri cand sunt create/updatate

d)Observability Patterns

13)Log Aggregation – AGGREGATOR DP

- Ai un serviciu de logging pentru toata aplicatia

14)Performance Metrics

- un serviciu de metrica care colecteaza statistici
- ai 2 moduri: push(serviciul ofera metrice serviciului de metrica) si pull(serviciul de metrica ia metrice de la serviciu)

15)Distributed Tracing

- cum urmaresti un request prin toate microserviciile apelate ca sa-l rezolvi?
- un serviciu
 - asigneaza la fiecare request extern un id unic
 - da id-ul la toate serviciile
 - include id-ul in toate log-urile
 - inregistreaza informatia(start/stop time) despre request-uri si operatiile efectuate

16)Health Check

- cand un serviciu ruleaza dar nu e capabil sa se ocupe de tranzactie

- fiecare serviciu trb sa aiba un endpoint care poate fi folosit ca sa vezi sanatatea aplicatiei
- e pentru load-balance

e)Cross-Cutting Concern Patterns

17)External Configuration

- cum eviti schimbarile in cod din cauza schimbarilor de configuratie?
- externalizezi toate configuratiile cu tot cu URL-urile endpoint si credentialele. Aplicatia ar trebui sa le incarce la start
sau in timpul executiei
- le poti folosi apoi ca proprietati de environment de exemplu

18)Service Discovery Pattern

- creezi un serviciu registru care sa mentina metadatele fiecarui serviciu producer.
- o instanta a serviciului trebuie inregistrata in registru cand e pornita si sa se delogheze cand e oprita.
- consumerul face query la registru ca sa afle locatia serviciului.
- ai client-side si server-side discovery

19)Circuit Breaker

- cand apelezi servicii din alte servicii e posibil ca unul sa fie down si sa ai erori in cascada. Dupa un anumit numar de erori,
serviciul va da pentru o perioada prestabilita fail instant
- dupa ce expira time-out-ul, intri in perioada de test, daca dupa un nr de request-uri totul e bine se revine la normal, daca nu
cade iar tot pt timeout

20)Blue-Green Deployment

- rulezi doua instante identice de medii de productie, blue si green. Green e instanta live iar blue e noua versiune a aplicatiei
- in orice moment doar un mediu e live si serveste toate cererile.

f)Altele

21) Bulkhead

Reactive programming : paradigma declarativa care se ocupa de fluxuri de date si cu propagarea schimbarilor.

Eveniment:

- e atomic
- inrudire
- comportament

Partea de comportament e cea mai importanta in proiectare si utilizare

Abordari arhitecturale

- Event-command pattern: serviciul stie endpoint-urile si API-urile celorlalte servicii si le apeleaza
- Event-first pattern: fiecare serviciu asculta pentru evenimente si emite zero sau mai multe eventuri dupa ce reactioneaza la
ce a primit

Caracteristici EDA(Event Driven Architecture)

- Comunicare broadcast
- reactie eficienta
- evenimente cu granularitate mica
- ontologie
- procesare evenimente complexe

Cele 4 planuri ale unei arhitecturi bazate pe fluxuri de evenimente(in paranteza exemplu pentru o plata)

- 1) business function(procesarea platii)
- 2) instrumentation plane(cati bani sunt transferati, cat de repede sunt procesati)
- 3) control plane(pattern-uri de start, stop, pauza, coordonare si autoscale)
- 4) operational plane(erori, urmarire si tratare, logging)

1)Business function: functionalitatea de baza care e construita. De obicei se ocupa cu procesarea datelor.

2)Instrumentation plane: captureaza metrice care dovedesc ca functia de business se comporta acceptabil. Pot fi folosite sa emiti alerte sau sa ajuti la automatizarea infrastructurii.

3)Control plane: cand ai nevoie sa opresti tot sistemul din procesare ca sa bagi un upgrade, si nu vrei sa se opreasca toate procesarile in curs

4)Operational plane: contine partea de automatizare, logica aplicatiei, jurnale de erori, jurnale de securitate, informatii pentru restaurare, cozile

de mesaje pierdute

Serverless computing e un model de executie in cloud in care un provider ruleaza serverul si manageriaza dinamic alocarea resurselor masinii. Pretul e bazat pe cate resurse consuma

aplicatia, fata de un pret stabilit dinainte.

If your PaaS can efficiently start instances in 20ms that run for half a second, then call it serverless"

Avantaje:

- incarcari mici
- decuplare maxima
- scalabilitate
- eficiente economic

Limitari:

- timp
- memorie
- tehnologie proprietara
- solutii publice
- securitate

Sisteme distribuite : Curs I

- C/C++/Java

- Spring

- IntelliJ

- Examen : Shell-uri laborator specifice SD (servicii si microservicii) + probleme PP

- sisteme strans cuplate : sute de metri intre componente (statii, procesoare, LAN sau BUS)
 - * supercomputere
 - * balbaielele automatistilor

- sisteme slab cuplate (intre cladiri) : INTRALAN, internetul este in centru
- (extra) : Intel baga ssd in core-ul de procesor

- framework-uri care sa ascunda problemele de dependenta realizata in background, referitor la memorie (ce tine de frecventa, tip, etc)

- **avantaj framework** : proiectare cu tot mai mult suport din spate (si in cazul sistemelor de operare)

- acestea (framework-urile) au condus treptat la arhitectura Java

- pana acum am facut aplicatii de tip MONOLIT (obiect si grad de dependeta mare intre obiecte) : daca-s multe obiecte : MARELE BULGARE DE NOROI (defect)

- aplicatie clasica : 50 de ani (cu tot cu multe modificari)

- alternativa mult mai buna care sa evite atata dependenta => servicii/microservicii : cresterea gradului de decuplare dintre componente

- **SUA/MSA** : MicroService Architecture

- **clasificarea lui Enslow** : (slide cu dreptunghi)
 - * control centralizat & date distribuite : internetul partial (unele componente din Internet respecta)
 - * calcul distribuit : foarte important in viitorul Cyperspace, cu toate ca este din ce in ce mai neglijat si inlocuit

- date distribuite : abordare de baze de date distribuita + ce va sa vina

- **sisteme de calcul distribuit** :

- * client-server (poate fi many to many si componentele pot avea alternativ ambele tipuri)

- * retele senzor : aplicatii in care omului ii este greu sa intervina (senzori pe animale, etc)

- **IOT/IOE** : dispozitive care preiau din mediul ambiant (bidirectional) informatii si se comporta ca un strat suplimentar pt CyberSpace

- * foarte diferit de retelele de senzori

- * IOE : internet of everything

- servicii suport la nivelul sistemelor de operare

- * servicii de retea

- * middleware (vezi daca asta e termenul) : pt aplicatii distribuite

- ** (extra) : basic : get post react (asta percep unii ca fiind SD)

- **HPC (High Performace Computing)** : supercomputere (tot parterul AC plin cu RAC-uri cu sisteme de calcul complexe, folosite de guvern, armata si sunt fucking scumpe)

- * modelari meteo, modelari ale scoartei terestre, this king of shit

- sistem distribuit != sistem de fisiere distribuit

- **SD** :

avantaje : * costuri reduse

- * modularitate si flexibilitate

- * fiabilitate (prin replicare : (!!!) sisteme K tolerante)

- * integritate

- * (!!!) referitor la propagarea informatiilor intre sisteme dependente in care se produc modificari : lazyapplication

* performanta : discutabil : definita strict in contextul problemei
implementate

dezavantaje : * intarziere : in cazul analizei de semnal : esantion trimis prin retea (=> intarziere)
ca sa ajunga intr-un loc in care se face analiza generala asupra intregului sistem

* lipsa unui timp global : sincronizare ceasuri pt coerenta date la
reconstructie

* nedeterminism : pisica lui Schrodinger (ex: TCP/IP : pachete ajung
aiurea si trebuie ordonate la destinatie , adica nu se poate preciza cum ajung acele pachete dar se
rezolva prin ordonare)

* securitate (foarte slaba)

- **sisteme k tolerante** : exemplu lift : pt functionare e nevoie de un singur cablu, dar se folosesc 3 pentru
siguranta (=> 3 tolerant)

- **arhitectura multinivel-multistrat** : mai multe niveluri care au mai multe straturi

- decuplarea functionalitatilor (!!!) : (mai multe in cursul II)

* SOLID

- **injectarea dependentelor** (mai multe in cursul II, deci stai chill pana saptamana viitoare) : (!!!)

* proxy

* apel de constructor

* se poate face manual/automat

Componente software

- granularitate software : OOP

- paradigma post OOP : CBSE (Component Based shit !!!)

- **procedural vs oop vs cop** : compozabilitate & interschimbarea

* COP : component oriented programming

- regula germana : creierul poate gestiona maxim 50 de obiecte :)

- **Java Beans (!!!)** : (clasa special construita) componentele/module din java (beans : componentele) comunica intre ele

- * serializabila

- * (!!!) cum impiedici serializarea ?

- **CORB (!!)**

- * broker

- * rol proxy (!!!)

- * rol design pattern bridge(!!!)

- **JAVA RMI (!!!)**

- * registru ce tine evidenta obiectelor (?)

- * ideea de stub (!!!)

- stub : o aplicatie n-are nevoie de toate obiectele ca sa lucreze asa ca aducem toate obiectele critice apoi aducem si restul si daca era nevoie de un obiect (de ex in cazul compilarii) se folosea un fel de proxy

(stub) care aducea si obiectul respectiv

- Serviciile

- * parte a economiei care desevertse o nevoie mai putin complexa (!= bun, adica sa iei un MAC se considera ca iei un bun, iar daca mergi cu el la reparat, ti se ofera un serviciu)

- * **servicii web** : colectie de API (folosire de elemente comune ?) care coopereaza la indeplinirea unei sarcini

- * **REST** (substandard HTTP care nu prea are securitate !!!)

- * **HTTP/HTTPS** : securitate "canci"

- * **SOAP (!!!)** - accent pe protocol (clasic, dar in prezent se foloseste doar cu HTTP)

- * tot ce are **REST/HTTP** in spate are securitate proasta

- * **servicii web** : publish find & bind

#####

Curs 1 – varianta 2

Un sistem distribuit e un sistem ale carui componente sunt localizate pe diferite computere legate intr-o retea si care comunica

si se coordoneaza prin trimiterea de mesaje unul altuia.

Caracteristici:

- concurenta componentelor
- lipsa unui clock global
- esecul independent al componentelor

Distributed control system: e un sistem de control pentru un proces compus din mai multe bucle de control in care controllere

autonome sunt distribuite prin tot sistemul, dar nu exista un supervizor central de control

Distributed database: o baza de date care e stocata in mai multe locatii fizice. Poate fi stocata pe mai multe calatoare in

aceeasi locatie fizica sau poate fi imprastiata intr-o retea de calatoare interconectate. Sunt alcatuite din parti slab cuplate

care nu impart componente fizice.

Un sistem de operare ia si pune in retea si transmite la aplicatia distribuita datele procesate.(forma primitiva)

O aplicatie distribuita e o aplicatie software care e stocata in principal in cloudsi care ruleaza pe mai multe sisteme simultan.

Sistemele ruleaza in aceeași retea si comunica unele cu altele intr-un efort de a realiza un anumit task.

O abordare ceva mai practica ar fi sa ai eventual servicii middleware care e un soft ce ofera functionalitati dincolo de cele

disponibile pe sistemul de operare. Face mai usor pentru developeri sa implementeze comunicare si I/O ca sa se poata concentra

pe scopul specific al aplicatiei lor.(le folosesti fiindca poti controla mai usor anumiți factori decat sa rescrii tu in SO)

Analiza SWOT(Strengths, Weaknesses, Opportunities and Threats)

S:

- costuri reduse(faptul ca poti folosi chestii de la alte firme, fara sa mai iei tu tot hardul, doar folosesti)
- modularitate si flexibilitate(in loc sa proiectezi aplicatii monolit, le poti imparti in module/servicii care pot fi extinse independent)
- fiabilitate si integritate(sistemul functioneaza fara defecte intr-un interval de timp si spatiu dat)
- performanta(poti lucra in paralel, si nu esti limitat de hard-ul de pe o singura masina)

W:

- lipsa cunostintelor despre starea globala(de obicei serviciile sunt stateless)
- lipsa unui timp global(pot aparea intarzieri datorate lipsei de sincronizare a clock-urilor la nivel global)
- nedeterminismul(nu stii mereu cui se trimite)
- comunicatiile(trebuie ca conexiunea sa fie buna pentru a nu scadea viteza sistemului din cauza transmisiei datelor)
- securitatea(trebuie securitate buna ca sa nu se afle date confidentiale trimise prin retea)

Servicii web(WS): o aplicatie modulara care poate fi descrisa, publicata, localizata si invocata pe web.

- fie un serviciu oferit de un dispozitiv altui dispozitiv, comunicand ambele prin www
- fie un server ce ruleaza pe un calator, asteapta request-uri si creeaza aplicatii web, care servesc la rezolvarea anumitor probleme prin intermediul web

De obicei ai un API(contract) pe care-l accesezi si folosesti de acolo ce ai nevoie

Poate fi descoperit de alte servicii si folosit.

WS sunt invocate folosind mesaje Simple Object Acces Protocol(SOAP). Aceste mesaje sunt codate in XML si transmise cu HTTP. Serviciul invocat se foloseste printr-un pattern de schimb(schema) de mesaje care e definit in WSDL(Web Service Description Language)

Poti avea model sincron(cand clientul se conecteaza, primeste un raspuns SOAP. E tip request-response) sau asincron(nu-l intereseaza pe client raspunsul. Trimite mesajul si serviciul il preia si trimite rezultatul cand e gata procesarea)

REST(Representational State Transfer)

- arh software

- poate utiliza SOAP pentru mesaje
- utilizeaza URI pentru a expune logica de afaceri. Poti mapa mai multe operatii pe acelasi URI
- preia securitatea protocolului de transport utilizat
- accepta diverse formate(XML, JSON, HTML)

SOAP(Simple Object Acces Protocol)

- protocol/set de standarde
- nu poate utiliza REST fiindca e protocol
- utilizeaza interfata serviciului(contractul) pentru a expune logica de afaceri
- defineste propriul standard de securitate
- lucreaza doar cu formatul XML

JEE:

- multinivel, multistrat
- cadru pentru dezvoltare rapida a aplicatiilor
- instalarea implica hard heterogen(sisteme cu mai multe tipuri de procesor/nuclee)

```
#####
#####
#####
```

CURS 2 – varianta 1

(continuare Curs I)

- Componente de afaceri
- * niste cursuri de economie(!!)
- * bean entitate : statefull, persistent

- * bean sesiune

- * bean bazat pe mesaje : bean sesiune + sistem de gestiune a mesajelor asincrone

- * servicii vs microservicii???

- Sistemul informational al intreprinderii

- * tot softul utilizat in intreprindere + soft de infrastructura

- * SAP???

- * cand proiectam NoSQL cu picioarele vor iesi costuri foarte mari

- * **containere** : locul in care instalam aplicatiile, iar aceste containere adauga tot ce trebuie ca aplicatia sa mearga (plugins, dependencies,etc) - se inghesuie totul in container

- **Ciclu de viata a unui bean de sesiune**

- * **fara stare** : dependency injection, daca exista si post construct callbacks daca exista

(Java are pointeri si acces la memorie!!nu se ocupa GC de tot)

- * **cu stare** : creare, dependency injection, post construct, init method sau ejbCreate

- * **orientat pe mesaje** : multe cozi (ca la stare) (vezi mai bine in curs!)

- **Spring**

- * dezvoltare usoara de aplicatii de intreprindere

- * scripturi de automatizare

- * linux pron???

- * **soft arch** : proiectare multilayer/tier si stabilirea tehnologiei folosite

- * stack-ul este particularizat in functie de proiect (gen full stack,etc) ???

- * **persistenta** : mysql, sqlite : oracle trebuie invatat

* **gradle si maven** (bucataria din spate) colectie de scripturi, nu framework

* **container principal (core)** : fabrica si gestiune de bean-uri]

* **scheduling** : planificare statica

* **load balancing** : planificare dinamica

* servicii extra : - web services

- security

- integration ???

- batch ???

- social : preluare date de la Fb si twitter pentru diverse

analize statistice

- mobile : (aplicatii care se muta)

- dynamic modules

- rich client

- .net

- flex

- roo : genereaza cod java

(exista tools care genereaza cod pe baza diagramei arhitecturale)

* extensii : - python

- stocare blob (azur???)

- persistenta bazata pe db4o si CouchDb??

- Decuplarea prin interfete

* incapsularea pe mai multe planuri

* respectare SOLID

* independenta cat mai buna prin separari

- Injectarea dependentelor

* operatie de scriere, verificare corectitudine, verificare destinatie si sursa corecte => fiecare se modeleaza separat, pe baza de obiecte si sa respecte si SOLID, adica fiecare dintre operatii sa poata fi folosita independent. putem si sa extindem clasa imbox cu aceste functionalitati dar pleaca SOLID instant

* injectare dependenta inseamna ca avem toate aceste servicii si facem ca email-ul sa depinda de ele.

* injectia : esentiala in modelarea grafurilor de obiecte, doar ca incarca obiectul deci este problematica intrucat modifica intr-un colt

dar strica in alt colt

* o alternativa buna era fabrica de obiecte (acum 40 de ani)

* control invers ???

* metadata : descriere sintetica a ceva ???

* XML

#####

CURS 2 – varianta 2

Spring e format din

1) Core Container:

- a) Beans
- b) Core
- c) Context
- d) SpEL(Spring expression Language)

2) AOP

3) Aspects

4) Instrumentation

5) Messaging

6) Test(pentru testare aplicatii)

7) WEB(MVC/REMOTING)

- a)WEB
- b)Web Socket
- c)Servlet
- d)Portlet

8) Data access

- a)Transactions
- b) OXM(Object XML Mapping)
- c) ORM(Object Relational Mapping)
- d) JMS(Java Messaging Service)
- e) JDBC(Java DB conectivity)

1)a)Beans

Spring se foloseste de **Inversion of Control(IoC)** care se mai numeste si dependency injection care e un proces unde obiectele isi defines dependentele(adica celelalte obiecte cu care lucreaza) doar prin argumente la constructor/argumente la o metoda factory(sau proprietatile care sunt setate pe instanta unui obiect dupa ce e construit/returnat de factory. Containerul injecteaza dependentele respective cand creeaza BEAN-ul. Nu mai controleaza bean-ul instantierea si localizarea ci containerul in sine.

Deci un bean e creat, asamblat si manageriat de IoC container.

loc se mai poate defini ca procesul in care un obiect doar isi defineste dependentele fara a le crea.

Trebuie doar sa configurezi cum trebuie IoC si nu te mai doare capul.

Acele configuratii sunt defapt alea cu @ de deasupra claselor si ii zici cum sa instantieze, configureze si asambleze obiectele in aplicatie.

Beanurile incep cu litera mica si sunt camelCase

Poti avea Injection de 2 tipuri:

- **constructor based:** class SimpleMoviePlaylist(private val movieFinder: MovieFinder), fara nicio adnotare speciala

- **setter-Based:** constructorul e fara argumente(dar poate avea si din alea), iar containerul va apela metode de set, pentru membrii clasei

(la kotlin poti pune doar lateinit var movieFinder: MovieFinder)

1)b) Core - Cuprinde IoC, AOP si altele

1)c)Context

Reprezinta containerele IoC

1)d) SpEL(Spring expression Language) - E un limbaj ce permite interogarea si manipularea obiectelor la runtime.

Functionalitati:

- expresii literale
- boolean si operatori relationali
- regex
- referinte la beanuri
- selectie de colectii

Il folosesti de obicei pentru parsari

Exemplu:

```
ExpressionParser parser = new SpelExpressionParser();  
Expression exp = parser.parseExpression("new String('hello world').toUpperCase()");  
String message = exp.getValue(String.class);
```

2)AOP(Aspect Oriented Programming)

Completeaza OOP-ul si ofera alt mod de a gandi structura unui program. Unitatea fundamentala la OOP e clasa iar la AOP e aspectul. E declarativ.

De exemplu o functie ar putea suna cam asa: "da log la toate apelurile functiilor ale caror nume incep cu set"

E pentru tranzactii, securitate, logging etc

4)Instrumentation

Iti permite sa adaugi cod binar la clasele java existente si compilate ca sa poti monitoriza performanta, diagnostica erorile etc. De obicei se

realizeaza prin logging.

5)Messaging

Lucrezi cu servicii care comunica prin mesaje pentru a fi slab cuplate. Iti poti alege ce fel de provider de mesaje vrei(JMS,RabbitMQ,Kafka)

pui dependentele necesare cu Maven/Gradle, si apoi inveti cum se utilizeaza fiecare adnotare si cum trimiti/primesti mesaje.

7)a)MVC

In spring request-urile HTTP sunt prelucrate de un controller(la care ii pui adnotatia @Controller)

@Controller

```
public class GreetingController {  
    @GetMapping("/greeting")  
    public String greeting(@RequestParam(name="name", required=false, defaultValue="World")  
String name, Model model) {  
        model.addAttribute("name", name);  
        return "greeting";  
    }  
}
```

@GetMapping spune ca ai de a face cu o cerere tip get

@RequestParam leaga valoarea din query-ul din URL de variabila din program

Dupa dai update la model si intorci rezultatul la view care este browser-ul.

7)b)WebSocket

Lucrezi cu mesaje

@Controller

```
public class GreetingController {  
    @MessageMapping("/hello")  
    @SendTo("/topic/greetings")  
    public Greeting greeting(HelloMessage message) throws Exception {  
        Thread.sleep(1000); // simulated delay
```



```
    return new Greeting("Hello, " + HtmlUtils.htmlEscape(message.getName()) + "!");  
}  
}
```

Daca se trimit mesaje la /hello, se apeleaza metoda greeting care trimite ce proceseaza din mesaj la /topic/greetings

7)c)Servlets

Servlet e un obiect care primeste request-uri si da raspunsuri pe baza lor

7)d)Portlets

E ca un servlet, doar ca daca faci acelasi request de 100 de ori(dai refresh la pagina) se face o singura procesare, si rezultatul e randat de 100 de ori la fel. Se mai proceseaza doar cand se produc schimbari.La servlet se proceseaza de fiecare data.

8)a)Transactions

O abstractizare consistenta pentru tranzactii.

8)b)OXM

Reprezinta procesul de conversie al unui XML in obiect si invers. Se mai cheama XML Serialization.

8)c)ORM

Conversia datelor intre tipuri de sisteme incompatibile folosind limbaje OOP(Java/Kotlin la Spring)

Ai foarte multa flexibilitate, poti face diverse straturi cu parti care nu depind de spring(slide 11)

Stiva spring:

- Spring MVC
- Service Manager
- DAO
- Hibernate(pentru DB)
- DB

Si totul e protejat folosind securitatea Spring

```
#####  
#####  
#####
```

CURS 3-4 – varianta 1

Sisteme Distribuite - Curs III

* NU SOAP, CI SOA!!! *

- **SOAP** : accelerarea ciclului de dezvoltare al aplicatiei

nu proiectam un sistem de intreprindere gandindu-l izolat;

nu proiectam o aplicatie fara sa avem in vedere inter-relationarea cu alte aplicatii
(eventual ale altor companii)

- **Structura intreprinderii** : dezvoltare incrementală pornind de la o idee a carei dezvoltare poate crește exponențial în funcție de mulți factori; necesită o serie de resurse

?? aplicatii monolit

- ideea refolosirii care a pornit odata cu dezvoltarea bibliotecilor

- la nivel de departament era un server in care erau toate aplicatiile si la nivel de intreprindere trebuia asigurata interoperabilitatea dintre toate aceste servere

- **Infrastructura tehnologica** : infrastructura hardware prin care sunt instalate aplicatiile (nas, ups, routere, aere conditionate, sisteme de operare, agenti, baze de date, sistem de gestiune a cozilor de mesaje, adaptoare, middleware(???), solutii de securitate, etc)

- **dezvoltare personala** : invata si hard si soft, deoarece va fi nevoie sa fie foarte bine inteleasa aceasta relationare pentru dezvoltarea aplicatiilor in contextului nivelului tehnologic din viitor

- **relatia afaceri** : abordare preemptiva! daca dezvoltam aplicatia pentru 100 de useri dar vor veni 10000, ce facem ? reprojectam si reimplementam? not a good idea!

- **relatia intre app si mediu de utilizare** : interoperabilitate stransa. cine raspunde de securitate ? tarile civilizate au standarde pe subiect

- **specificul proiectului** : rezulta din maniera de proiectare proiectarea e influentata si de arhitectura ei exterioara citire standard apoi alte vrajeli !

- **model de proiectare** : design patterns : solutie validata empiric pentru specificul unei probleme

- **standard de proiectare** : influentata mult de ideea si principiul de afaceri se pot adapta si la nivel particular suporta imbunatatiri continue standard proiectare != standard industrial

- **calcul orientat pe servicii** : - platforme sau software pt calcul distribuit

- crearea entitati logice cu grad mare de izolare care pot fi utilizare si reutilizate intre ele pentru a servi proiectarii unei arhitecturi

- serviciu SOAP != serviciu din SO

- un IT-ist foarte bun intelege problemele de business (partea economica!)

- costurile de IT sunt asa mari, serviciile foarte complexe carora li se aplica toate principiile contabilitati : beneficii, cat costa mentenanta, intretinerea, actualizarea, instalarea, dezvoltarea

- grave probleme legale si de confidentialitate

- **serviciu** : implementarea unui program sau a unei aplicatii care rezolva o anumita sarcina, avand in vedere o serie de principii SOAP sau de intreprindere

componenta/modul nativ utilizata de mai multe aplicatii (monolit???)

- in business, companiile mari ajung sa aiba piata stratificata si au o serie de clienti fructuanti si atunci se bat prin oferta personalizata ca sa ii aduca intr un anumit strat

- **SWOT SOA** : A : * fiabilitate

* scalabilitate

D : * incarcare a sistemului (continuu, o aplicatie gen facebook : cand noi dormim, americanii nu dorm)

* costuri initiale mari

* gestiune complexa a serviciilor

- **monolit** : am inlocuit o durere mare de cap cu mai multe dureri mici de cap (se proiecteaza prost!)

- **SOA** : orchestrare : administrare si gestionare a mesajelor intervenite in interoperare

(la ghiseu, cand las un plic care este distribuit in alta parte, in spate)

- **inventarul serviciilor** : ca o carte de telefonie : colectie logica, care poate reprezenta o intreprindere sau o componenta de afaceri intr o intreprindere

structura de tip top down

- **agnostic** : serviciu total independent

- **oop -> component -> servicii -> microservicii -> serverless computing**

- **serviciu candidat** : rafinare (la implementarea propriu zisa) : proiectare, asamblare, implementare, modificare, schimbari in contract, etc

prima propunere asa cum e vazuta de intreprindere

- **3 maniere de implementare a abordarii SOA :**

- JBeans

- servicii web (o va matura serverless computing)

- bazata pe componente

* **bazata de componente** : incapsulare in serviciu (bataie de joc referitor la resurse)

* **servicii web** : bazat pe scheme XML si cu probleme grave de securitate

* **servicii REST (SOA)** : simplitate, scalabilitate, reutilizare, pt chestiuni simple

(SOA != REST, REST este un <modul?> din SOA)

- abstractizare serviciu

- **stateless service** : nu inseamna ca nu are stare, ci ca starea lui este salvata inafara

- **Caracteristici SOA :**

* combinatie de framework-uri (probleme de securitate asa ca mai bine o dezvolti de la 0)

* BD federative ???

- * strat uniform care ascunde serviciile din spate si stratificarea
- * DevOps ???
- * DevSecOps ???
- * injectia dependentelor : nu modific serviciul primar ci mai adaug alte servicii pe care le injectez in serviciul primar (SOLID)
- * **ROY/ROI** ????
- * business driven
- * centrat pe intreprindere

- Tipuri comune de SOA :

- arhitectura serviciului
 - arch compunerii de servicii
 - arch catalogului de servicii
 - arch bazata pe servicii
- * granularitati diferite (???granularitate, incearca sa intelegi bine ce e!)

- VSDL, XSD

- SLA

- serviciul nu e o clasa mai mare !!

```
#####
#####
#####
```

CURS 3-4 varianta 2

SOA(Service Oriented Arhitecture)

Avem:

- program software

- arhitectura tehnologica
- infrastructura tehnologica

Fiecare tip de arhitectura are propriile specificatii

Component architecture->

- >application architecture(in care grupezi mai multe componente)->
- >integration architecture(in care lipesti mai multe aplicatii)->
- >enterprise tehnology architecture(in care pui de toate + DB si sist legacy)

RPC(remote procedure call) apelezi o procedura de pe alt calculator(de obicei printr-o retea) ca si cum ar exista local

BPM(Business process modeling) e activitatea de reprezentare a proceselor unei intreprinderi astfel incat procesul curent sa poata fi analizat

imbunatatit si automatizat.

EAI(Enterprise application integration) reprezinta folosirea soft-ului si a principiilor arhitecturale ale sistemelor de calatoare pentru a

integra un set de aplicatii la nivel de intreprindere

Un serviciu are mai multe capabilitati(ce poate face serviciul respectiv)

SWOT SOA

S:

- reutilizarea serviciilor(trebuie sa aiba ROI(Return of Investment) mare, sa ia parte la mai multe lucruri decat cele pentru care au fost

proiectate initial)

- usor de mentinut(sunt documentate si n-ar trebui sa fie greu de modificat)
- Independente de platforma(merg pe toate platformele(cam cum e JVM))
- Disponibilitate (ai nevoie sa apelezi un serviciu de la o anumita firma, nu conteaza ca e 1 noaptea sau 2 dupa amiaza, tot la fel merge)

- Fiabilitate (sistemul sa fie disponibil)
- Scalabilitate (serviciile sunt participanti activi la compozitii)

Un serviciu poate fi pe rand, producator sau consumator. Cand e producator, proceseaza ceva si trimite mai departe pentru cine are nevoie, iar

cand e consumator, preia rezultatele de la altii si le foloseste.

Compunerea serviciilor: se apeleaza unele pe altele pentru a indeplini o functie/scop mai mare. Putem avea:

- **orchestrare**: ai un serviciu coordonator, care apeleaza intr-o anumita ordine alte servicii
- **coregrafie**: serviciile se apeleaza unul pe altul(un fel de Chain of Responsibility) pentru a obtine rezultatul dorit

Un serviciu este slab cuplat prin faptul ca comunica prin mesaje, si serviciile din exterior stiu doar de contractul lui(care e interfata, adica

operatiile disponibile)

Cele 4 tipuri comune de SOA:

- la nivel de serviciu
- la nivel de compozitie
- la nivel de inventar
- la nivel de intreprindere

Terminologie:

Design Characteristic:

- o proprietate a unui program software sau **Tehnology Architecture**(TA) care rezulta din modul in care a fost proiectat

- poate fi orice calitate concreta, cum ar fi faptul ca programul e alcatuit din componente, ca functiile au granularitate fina sau aspra

Design Principle:

- practica acceptata de industrie care are un anumit scop
- paradigma SOA e compusa din principii de design care sunt aplicate impreua pt a atinge obiectivele orientarii pe serviciu

Design Pattern:

- o solutie demonstrata la o problema de design, documentata intr-un format consistent

Design Standard:

- conventii de design particularizate individual de organizatie ca sa livreze solutii in concordanta cu telurile de business specifice organizatiei

Service Orientation:

- paradigma de design pentru crearea unitatilor solutiei logice care sunt proiectate individual ca sa fie utilizate repetat si colectiv in

scopul realizarii unor obiective

- unitatile se numesc SERVICII

Serviciu:

- e o unitate logica asupra careia s-au aplicat intr-o masura semnificativa principiile proiectarii service-oriented

Service Capability:

- fiecare serviciu are propriul context functional distinct si e compus dintr-un set de functii numite capabilitati legate de context

- cercelele alea din desenul serviciului

- nu influenteaza cum e implementat serviciul

Service Consumer:

- cand un program invoca si interactioneaza cu un serviciu se numeste service consumer
- termenul se refera la rolul temporar luat la runtime de un program atunci cand e angajat cu un serviciu pentru schimb de date

Service Provider:

- serviciul invocat de consumer

Service Composition:

- agregare de servicii compuse colectiv pentru a automatiza un anumit task/proces de business

- ca sa avem compozitie, e nevoie minim de 2 servicii participante si un initiator al compozitiei. Altfel e doar schimb

Service Inventory:

- colectie de servicii complementare, standardizate, independente si guvernate, aflata in limite care reprezinta o afacere sau un segment al afacerii
- cand ai mai multe inventare termenul devine domain service inventory

Principiile service oriented:

1)Standardized Service Contract

Serviciile din acelasi inventory sunt in conformitate cu aceleasi standarde de design

2)Service Loose Coupling

Contractele serviciilor impun cerinte de cuplare scazute pentru consumator si sunt la randul lor decuplate de mediul inconjurator

3)Service Abstraction

Contractele serviciilor contin doar informatii esentiale si informatiile despre servicii sunt limitate doar la ce e publicat in contract

4)Service Reusability

Serviciile contin si exprima logica agnostica(nu conteaza contextul in care sunt chemate) si pot fi considerate resurse enterprise reutilizabile

5)Service Autonomy

Serviciile exercita un nivel ridicat de control asupra mediului de executie underlying la runtime

6)Service Statelessness

Serviciile minimizeaza consumul de resurse prin amanarea managementului informatiilor despre stare cand e necesar

7)Service Discoverability:

Serviciile sunt suplimentate cu date meta de comunicatie prin care pot fi descoperite si interpretate eficient(cum e docker, ai un registru si le iei de-acolo)

8)Service Composability

Serviciile sunt participanti efectivi la compozitie, indiferent de marimea si complexitatea compozitiei

Caracteristici de baza SOA:

a)Business-driven:

- TA e aliniata cu arhitectura curenta de business. Contextul e apoi mentinut constant asa incat TA sa evolueze in tandem cu businessul de-a lungul timpului.

- TA se tot indeparteaza de Business pana in punctul in care trebuie gandita de la 0 si de aia trebuie sa le tii aproape

b)Furnizor neutru:

- Modelul arhitectural nu e bazat strict pe o platforma furnizor, poti combina mai multe tehnologii, sa le inlocuiesti pe unele ca sa maximizezi realizarea

necesaritatilor afacerii in mod constant

- ai mai multa libertate la implementare

c)Enterprise-centric:

- Scopul arhitecturii reprezinta un segment important al intreprinderii, permitand reutilizarea si compozitia serviciilor, permitand solutii orientate serviciu in

favoarea celor traditionale

- nu iti imparti serviciile pentru anumite lucruri, le pui la dispozitie pentru toti din firma

d)Composition-centric:

Arhitectura suporta mecanici de agregare repetata a serviciilor, permitand acomodarea schimbarilor constante prin asamblare agila a compozitiilor de servicii

SOA Design Patterns:

Inventory Boundary(identifica granitele inventarului)

1)Enterprise Inventory

- Servicii pentru multiple solutii pot fi proiectate pentru livrare intr-un inventory arhitectural standardizat, la nivel de intreprindere unde pot fi recompose repetat si liber

- recomandat pentru organizatii mici/medii

2)Domain Inventory

- serviciile pot fi grupate in inventare de servicii specifice domeniului, fiecare putand fi guvernat, standardizat si detinut independent

- la intreprinderile mari nu mai merge 1) fiindca ar dura prea mult analiza

Inventory Structure(structura high-level si complexitatea generala a unui inventory e data si influenteaza nevoile pe care arhitectura trb sa le indeplineasca)

3)Service Normalization

- inventarul trebuie proiectat cu accent pe alinierea limitelor serviciilor
- serviciile fiind normalizate se elimina riscul ca 2 servicii sa aiba limite functionale suprapuse

4)Logic Centralization

- Accesul la functionalitatea reutilizabila e limitata la serviciile agnostice oficiale
- ca sa nu ai functionalitate redundanta

5)Service Layers

- inventar structurat in 2 sau mai multe straturi logice, fiecare fiind responsabila pt a abstractiza logica pentru un tip functional comun
- minim 2 layere, unul cu logica agnostica(pentru mai multe scopuri), celalalt cu logica non-agnostica(pt un singur scop)

Inventory Standardization(Standardele se stabilesc pt a asigura o baza comuna pentru interoperabilitatea intre servicii)

6)Canonical Protocol

- arhitectura stabileste o singura tehnologie de comunicare ca mediu primar in care serviciile pot interactiona
- doar la comunicarea intre servicii, nu si in interiorul unui serviciu

7)Canonical Schema

- modelele de date pentru seturi de informatii comune sunt standardizate de-a lungul contractelor in limitele inventarului
- schema e ce treci tu de exemplu intr-un fisier xsd(XML schema definition) ca sa ai acelasi format de date intre servicii

Logical Inventory Layer(ai logica utilitara, agnostica si non-agnostica)

8)Utility Abstraction

- un nivel de servicii dedicat procesarilor utilitare, ce ofera servicii utilitare pentru a fi folosite de alte servicii din inventar
- nu e logica de business, ci doar logica utilitara(de exemplu libraria matematica)

9)Entity Abstraction

- un nivel de servicii business agnostice poate fi stabilit
- dedicat serviciilor ale caror context functional se bazeaza pe entitati existente de business
- separi serviciile agnostice de cele non-agnostice
- o faci doar daca nu se schimba des specificatiile

10)Process Abstraction

- un nivel parinte de business dedicat
- stabilit pentru a suporta administrarea independenta si a face serviciile task(non-agnostice) potentiale resurse enterprise

Inventory Centralization

11)Process Centralization

- logica ce reprezinta numeroase procese de business poate fi livrata si administrata intr-o locatie centrala
- ai o platforma care orchestreaza toata logica de business

12)Schema Centralization

- selecteaza scheme care exista ca parti fizice separate ale unui contract si sunt partajate intre mai multe contracte
- elimina schemele redundante
- ai WSDL(Web Service Description Language) care e validat de scheme

13)Policy Centralization

- Politici globale sau de domeniu pot fi izolate si aplicate mai multor servicii
- elimini redundanta
- ariile adresate de politici: securitate, conditii de transfer, serie de proprietati QoS(Quality of Service)

14)Rules Centralization

- Stocarea si administrarea regulilor de business se afla intr-o extensie arhitecturala dedicata de unde pot fi accesate si mentinute
- Poti crea un serviciu central pentru a se ocupa de crearea, modificarea, aplicarea si primirea regulilor de business

Inventory Implementation

15)Dual Protocols

- arhitectura inventarului e proiectata sa suporte servicii bazate pe protocolul principal si secundar
- un singur protocol poate sa nu fie capabil sa acomodeze toate necesitatile serviciului, si de aia mai introduci unul
- trebuie folosita in moderare

16)Canonical Resources

- Infrastructura si arhitectura pot fi echipate cu resurse comune si extensii care pot fi utilizate repetat de diferite servicii
- ex de resurse: DB, directoare, extensii de securitate

17)State Repository

- date legate de stare pot fi scrise temporar si apoi sa fie luate dintr-un repo dedicat
- reduci consumul de memorie asociat cu retinerea informatiilor de stare

18)Stateful Services

- Datele legate de stare sunt organizate si stocate de servicii stateful utilitare
- starea e impartita la mai multe servicii utilitare fiind mai usor de organizat

19)Service Grid

- Datele de stare sunt intarziate catre o colectie de servicii stateful care formeaza un grid ce ofera scalabilitate, toleranta la esec
- prin replicare a memoriei si redundanta suportand infrastructura
- eviti riscul de a pierde date in partile super folosite

20)Inventory Endpoint

- capabilitatile relevante sunt abstractizate intr-un serviciu de capat care se comporta ca un punct de intrare oficial in inventar si e

dedicat unui set de consumatori externi

- oferi doar anumite functionalitati din inventar, nu acces la tot inventarul
- nu fac parte din inventar si nu participa la compozitii

21)Cross-Domain Utility Layer

- un nivel utilitar comun poate fi stabilit, intinzandu-se intre 2 sau mai multe inventare de domeniu
- ai redundanta inutila la logica utilitara
- poti imparti in logica utilitara specifica domeniului, si logica utilitara inter-domeniu

Inventory Governance

22)Canonical Expression

- contracte standard folosind conventii de numire
- contractele pot exprima capabilitati similare in diferite moduri, si nu vrei asta

23)Metadata Centralization

- metadatele serviciilor pot fi publicate central intr-un registru ce ofera un mijloc formal de a inregistra si descoperi serviciile(docker)

24)Canonical Versioning

- regulile de versionare a contractelor si expresia informatiilor de versionare sunt standardizate in limitele inventarului
- poti avea versiuni de contracte diferite in inventar si asta cauzeaza probleme

Service Identification

25)Functional Decomposition

- poti sparge problema intr-un set de probleme mai mici si solutia la fel
- eviti aplicatiile monolit

26)Service Encapsulation

- Logica solutiei e incapsulata de un serviciu care e pozitionat ca o resursa enterprise capabila de a functiona dincolo de limitele pentru
care a fost conceput initial
- trebuie sa proiectezi pentru mai mult de un singur mediu pentru a nu fi limitat

Service Definition

27)Agnostic Context

- Izoleaza logica care nu e specifica unui singur motiv, in servicii separate cu logica agnostica

28)Non-Agnostic Context

- Logica non-agnostica potrivita pentru incapsularea serviciilor poate fi localizata in serviciul care e gazduit ca membru oficial al unui inventar
- pot fi controllere/initiatori ai compozitiilor

29)Agnostic Capability:

- Serviciul agnostic e partitionat intr-un set de capabilitati bine definite care adreseaza preocupari comune care nu-s specifice niciunei probleme.

Prin analiza se rafineaza si mai mult

Service Implementation

30)Service Facade

- o componenta fatada e folosita pentru a abstractiza o parte din serviciile cu potential negativ de cuplare
- ajuta serviciul principal prin niste procesari suplimentare.

31)Redundant implementation

- Serviciile reutilizabile pot fi livrate prin implementari redundante sau cu suport la erori
- eviti single point of failure

32)Service Data Replication

- serviciile pot avea propriile DB cu replicare pentru datele partajate

33)Partial State Deferral

- chiar daca serviciile trebuie sa ramana stateful, o parte din datele de stare pot fi intarziate temporar
- tii doar o parte din stare in memorie

34)Partial Validation

- un consumer poate fi proiectat sa valideze doar partea relevanta a datelor

35)UI Mediator

- Stabileste un mediator care e responsabil doar cu asigurarea interactiunii la timp si feedback de la UI si prezentare

Service Security

36)Exception Shielding

- exceptia unsafe e inlocuita cu una care e safe din proiectare si data la consumer

37)Message Screeing

- serviciul e echipat sau suplimentat cu rutine de ecranare care pp ca orice input e daunator pana se dovedeste contrariul

38)Trusted subsystem

- serviciul e proiectat pentru a folosi credentiale proprii pentru autentificare si autorizare legata de resursele backend in numele consumerului

39)Service Perimeter Guard

- un serviciu intermediat e stabilit ca perimetru al retelei locale ca un punct de securitate pentru orice consumer extern care trb sa interactioneze
cu serviciile interne

Service Contract Design

40)Decoupled Contract

- contractul serviciului e fizic decuplat de implementare si pot varia independent

41)Contract Centralization

- accesul la serviciu e limitat de contract, fortand consumerii sa evite cuplarea cu implementarea

42)Contract Denormalization

- Contractele pot include o cantitate masurata de denormalizare, permitand mai multor capabilitati sa exprime redundant functionalitati de baza in
diverse moduri pentru diferite tipuri de consumatori

43)Concurrent Contracts

- Mai multe contracte pentru acelasi serviciu, fiecare pentru un tip de consumer

44)Validation Abstraction

- Validarea granulara si regulile pot fi abstractizate de contract, reducand granularitatea serviciului si crescand longevitatea contractului

- contractele cu multe constrangeri devin invalide cand se petrece o schimbare

Legacy Encapsulation

45)Legacy Wrapper

- un serviciu wrapper non-standard poate fi inlocuit sau impachetat intr-un serviciu standard care incapsuleaza si poate elimina detaliile tehnice legacy din contract

- impachetezi sistemul legacy intr-un serviciu

46)Multi-Channel Endpoint

- un serviciu intermediar e proiectat sa incapsuleze sistemele specifice canal si sa expuna un singur contract standard pt consumatori specifici canal multipli

- se refera cand proiectezi de exemplu pentru pc/telefon ca ai mai multe feluri de sist legacy, si vrei sa ai un singur serviciu care comunica cu toate device-urile

47)File Gateway

- logica de procesare bidirectionala intre sistemele legacy si servicii ca sa poata comunica, fiindca nu se pot invoca unele pe altele

Service Governance

48)Compatible Change

- unele schimbari ale contractului pot fi compatibibile inapoi, evitand impactele negative ale consumatorilor

49)Version Identification

- informatiile de versionare referitoare la schimbari compatibile si incompatibile pot fi exprimate ca parte a contractului, pentru motive de comunicare si intarire

- minor version nu afecteaza userii, major da

50)Termination Notification

- Contractele pot fi proiectate sa exprime informatii de terminare pentru consumarii umani si programati

- "suportul pentru aceasta versiune se va termina incepand cu data..."

51)Service Refactoring

- Contractul e pastrat pentru a mentine dependentele cu consumerul, dar logica serviciilor underlying e refacuta

- poti avea serviciul la v2 si contractul la v1

52)Service Decomposition

- un serviciu se poate descompune in 2 sau mai multe servicii mai fine

53)Proxy Capability

- contractul original e pastrat, chiar daca logica de dedesubt e separata, prin transformarea definitiei capabilitatilor intr-un proxy

- adica stie contractul ce sa apeleze din serviciu

54)Decomposed Capability

- serviciile supuse riscului de a fi supuse descompunerilor ulterioare pot fi echipate cu o serie de capabilitati granulare care pot facilita mai usor descompunerea

55)Distributed capability

- partea underlying a serviciului e distribuita, permitand implicit ca o capabilitate cu necesitati unice de procesare sa fie separata, in acelasi timp continuand sa fie reprezentata de acelasi contract

- logica de capabilitate e in alta parte de obicei

Capability Composition

56)Capability Composition

- Cand logica necesara e in afara, capabilitatea din serviciu e proiectata sa compuna una sau mai multe capabilitati in alte servicii

57)Capability Recomposition

- Capabilitatile serviciilor agnostice pot fi proiectate pentru a fi invocate repetat pentru a suporta mai multe compozitii care rezolva multiple probleme

- folosirea unui serv agnostic ca sa rezolvi o singura problema, nu creste gradul de reutilizare

Service Messaging

58)Service Messaging

- serviciile pot fi proiectate sa interactioneze prin mesaje care reduce conexiunile persistente si necesitatile de cuplare

59)Messaging Metadata

- continutul mesajelor poate fi suplimentat cu metadate specifice actiunii care sunt interpretate si procesate separat la runtime

60)Service Agent

- service agents pot fi proiectati sa raspunda automat unor conditii predefinite fara invocarea printr-un contract
- event driven

61)Intermediate Routing

- caile mesajelor pot fi determinate dinamic prin logica intermediara de rutare

62)State messaging

- in loc sa retii in memorie, stochezi temporar in mesaj

63)Service Callback

- un serviciu poate necesita ca consumerii sa comunice asincron cu el si sa ofere o adresa callback unde serviciul sa poata trimite raspuns la mesaje

64)Service Instance Routing

- serviciile ofera un id de instanta impreuna cu informatii de destinatie intr-un format standard care fereste consumatorul ce a avea sa recurga la logica custom
- ca sa mentii starea in caz ca ai nevoie de un schimb de mesaje

65)Async Queuing

- un serviciu poate schimba mesaje cu consumerii printr-un buffer intermediar, permitand serviciilor sa proceseze mesajele independent prin ramanerea decuplat temporar

66)Reliable Messaging

- un mecanism de incredere intermediar e introdus in arhitectura inventarului, asigurand ca livrarea mesajelor e garantata
- pentru medii unreliable

67)Event Driven Messaging

- consumerul se stabileste ca subscriber al serviciului. Serviciul notifica automat eventurile relevante la toti subscriberii

Composition Implementation

68)Agnostic Sub-Controller

- logica reutilizabila agnostica inter-serviciu e abstractizata sau e facuta disponibila printr-o capacitate agnostica de sub-controller, permitand subsetului sa fie recompus independent

69)Composition Autonomy

- Toti participantii la compozitie pot fi izolati sa maximizeze autonomia compozitiei ca întreg

70)Atomic Service Transaction

- Actiunile de runtime pot fi impachetate într-o tranzitie cu capacitati de rollback care reseteaza toate actiunile si schimbarile daca parintele nu poate fi completat cu succes

71)Compenstating Service Transaction

- Rutine de compensare sunt introduse, permitand exceptiilor runtime sa fie rezervate cu lock redus pe resurse si memorie redusa
- poti consuma prea multe resurse folosind 70)

Service Interaction Security

72)Data Confidentiality

- mesajele sunt criptate

73)Data Origin Authentication

- un mesaj poate fi semnat digital si se poate verifica originea si daca nu a fost alterat pe parcurs

74)Direct Authentication

- trebuie ca utilizatorii sa dea credentiale in mesaj ca sa foloseasca serviciul

75)Brokered Authentication

- Un broker de autentificare ca un depozit centralizat de identitati, isi asuma responsabilitatea pentru autentificarea userului si emiterea unui token pe care consumerul il poate folosi sa acceseze serviciul

Transformation Patterns

76)Data Model Transformation

- O tehnologie de transformare a datelor poate fi incorporata sa converteasca datele între scheme diferite

77)Data Format Transformation

- Logica de transformare intermediara a formatului datelor trebuie introdusa ca sa traduci dinamic dintr-un limbaj in altul

78)Protocol Bridging

- Logica de bridge e introdusa pentru a permite comunicarea intre diferite protocoale prin convertirea dinamica a unui protocol in altul

Enterprise service bus(ESB)implementeaza un sistem de comunicare intre aplicatii soft in SOA

Dirijarea dinamica se refera la trimiterea spre anumite servicii in functie de continutul mesajului.

Filtrul e fix filtrarea mesajelor din Design Patterns

#####

Curs 5 – varianta 1

Cloud

Serviciile Amazon

- au sisteme de distributie LINUX
- Elastic Compute Cloud
- Simple Storage Service
- SimpleDB
- CloudFront

Windows Azure

- aplicatii executabile in Cloud
- probleme de securitate si stabilitate

Cloud

- esenta cloud-ului : virtualizare (???)

Virtualizare

- Translare binara : doar cand kernelul este apelat (???)
- abstractizarea sistemului de operare, aplicatiilor si zonelor de pastrare a datelor sau a retelei de comunicatii fata de suportul hardware si software care permite acest proces

- Virtualizare Red Hat (???)

- Tipuri de virtualizare

- completa

- para virtualizare
- hibrida

- **Virtualizare completa** : simulare completa a unui hardware sau software

- **pe soft** (translate binara) si emuleaza hardul folosind instructiuni soft compatibile
- **pe hard** : se bazeaza pe suportul nativ de hardware, se pot executa direct pe procesor
- **hypervisor** : (???)

- **Hypervisor de tip 2**

- masina virtuala peste sistem de operare
- incarcare destul de mare (pt o comanda se trece printr un lant de apeluri specifice masinei virtuale, dar si pe gazda pt compatibilitate si executie)

- **Paravirtualizare**

- XEN (???)
- peste sistemul gazda, nu se instaleaza un sistem de operare oaspete full, ci modificat astfel incat sa se potriveasca cu hardware ul; aceasta se face cu niste drivere instalate in spate de masina virtuala
- elimina incarcarea excesiva

- **Virtualizare la nivel de sistem de operare**

- Oracle Solaris
- Docker (??? pt tema)

- **Terminologie si concepte Cloud**

- Hiperconvergenta ???
- Centru de calcul cu patru straturi
- Platforma hip

- **Arhitectura Cloud**

- n level n tier
- echilibrarea incarcarii
- multe cozi

- **Nivel de prezentare Cloud**

- pagini web cu servicii disponibile
- infrastructura elastica
- multiplicare verticala : se poate muta pe alta masina
- multiplicare orizontala : se muta si pe alte masini si se distribuie resursele pt eliminarea incarcarii (trebuie facuta jurnalizare)
- nivel 2

#####

Curs 5 – varianta 2

Cloud computing: disponibilitate la cerere a resurselor computerelor, in special a stocarii datelor(cloud storage) si putere computationala fara management direct si activ din partea userului.

Norii pot fi limitati la o singura organizatie(enterprise clouds) sau disponibile la mai multe(public clouds)

E la intersectia a 4 domenii

- Hardware

1) Hardware virtualization: e virtualizarea computerelor ca platforme complete hardware

Maschine virtuale de exemplu care sunt programe ce simuleaza o intreaga platforma hardware cu componente etc.

2) Multi-core chips: un procesor integrat cu 2 sau mai multe utiati de procesare separate numite core-uri/nuclee fiecare citind si

executand instructiunile programelor ca si cum computerul ar avea mai multe procesoare.

- Internet Tehnologies

1) SOA

2) Web 2.0

3) Web Services

4) Mashups: o pagina/aplicatie web care foloseste continut din mai multe surse pentru a crea un nou serviciu afisat intr-o singura pagina web

De exemplu poti combina pozele tale de pe site cu un Google map ca sa ai o mapa custom.

- **Automation**(nu e specificata in curs, doar i-am pus eu nume)

1) Automatic Computing: caracteristicile de manageriere propriie a unor resurse distribuite de computing, adaptandu-se la schimbari imprevizibile

ascunzand complexitatea intrinseca operatorilor si userilor

2) Data Center Automation: proces prin care rutinele de workflow si procesele unui data center sunt executate si manageriate fara interventia umana

- Distributed Computing

1) Utility Computing: un model de aprovizionare cu servicii in care un provider face resursele de computing si infrastructura disponibila clientului

dupa necesitati si il taxeaza in functie de consum.

2) Grid Computing: folosirea la scara larga a resurselor computerelor pentru a atinge un tel comun. Nodurile grid-ului executa o aplicatie/task diferit.

Traditional vs IaaS(Infrastructure as a Service) vs PaaS(Platform as a Service) vs SaaS(Software as a Service)

In mod normal trebuie sa manageriezi

a)aplicatiile

b)data

c)runtime

d)middleware

e)OS

f)Virtualizarea

g)Serverele

h)Stocarea

i)Networking

Cu **IaaS**, producatorul manageriaza de la OS in jos, si tu de la OS in sus(mult mai convenabil)

Cu **PaaS**, producatorul manageriaza de la runtime in jos, tu doar aplicatiile si data

Cu **SaaS** Se ocupa provider-ul de tot

Alta definitie Cloud Computing: model de plata functie de utilizare care permite accesul pe baza de retea, la cerere, convenabil, disponibil, la o grupare de

resurse de calcul configurabile(retele, servere, stocare, aplicatii)

Aceste servicii pot fi oferite rapid, cu efort de administrare minimal sau cu interactiune minimala cu furnizorul de serviciu

CLOUD

Caracteristici de baza:

- auto-service la cerere
- acces oriunde la retea
- grupare resurse independente de locatie
- plata dupa cat consumi

Tipuri de cloud:

- public external(public clouds)
- private internal(enterprise clouds)

FaaS(Functions as a Service)

E ca **PaaS** dar n-ai nevoie de un server care sa ruleze in spate mereu. Ai si mecanisme de caching si ovrall iesi mai ieftin

Exemple:

- **API Gateway**
- **Funciton Watchdog**
- **Prometheus**
- **Swarm**
- **Kubernetes**
- **docker**

XaaS(Orice ca Serviciu)

- Storage
- DB
- Communication
- Network
- Monitoring
- Testing
- HPC
- Human
- Process
- Information
- Identity
- Application
- Integration
- Governance
- Security
- Backup

Servicii Amazon

- Elastic Compute Cloud
- Simple Storage Service
- Simple Queue Service
- SimpleDB
- CloudFront

Windows Azure

- Live Services
- .NET Services

- SQL Services
- SharePoint Services
- Microsoft Dynamics CRM Services

Terminologie si concepte specifice cloudurilor

- Tier 4 data center:

99.995% uptime/an

2N+1 infrastructura full redundanta

96 ore protectie impotriva penelor de curent

26.3 minute de cadere anuala

- Hiperconvergenta

Reteaua de stocare si abstractizarile soft sunt implementate virtual in software nu in hardware

- Platforma hiperconvergenta

Alcatuit din:

- Presentation Tier

1)Load Balancer

2)Elastic Load Balancer(care e o coada)

3)Presentation Application Component care comunica cu Business Logic Tier

- Stateless Component

- UI Component

- Business Logic Tier

1)Elastic Queue

2)Business Logic Application Component

- Stateless Component
- Processing Component

- Data Tier

- 1)Elastic Queue
- 2)Data Access Component
- 3)Storage Offerings

Tipuri de scalari:

- scale-up/vertical Scaling

cererile vin printr-un front-end, sunt preluate de un Load Balancer si apoi serverul
hosteaza

o copie completa a aplicatiei

Scalare prin cresterea resurselor(CPU, RAM) a fiecarui host

- scale-out/Horizontal Scaling

scalare prin adaugarea mai multor servere

```
#####
#####
#####
```

CURS 6 – varianta 1

Microservicii

Persistenta

- jdbc blocant
- imutabilitate Critic pt microservicii

Arhitecturi bazate pe tratare de evenimente

- nu este nevoie de mapare relationala a obiectelor
(ev. sunt inregistrate)

CURS 6 – varianta 2

Arhitecturi bazate pe tratare de evenimente

Avantaje:

- nu e nevoie de mapare relationala a obiectelor
- cu fiecare schimbare externa capturata ca eveniment, starea curenta poate fi repetata(pur si simplu repeti evenimentele, ca la pattern-ul Command)
- asigurarea persistentei se face fara modificari
- Deci, nu mai ai nevoie de un SGBD relational

Monolit(o singura unitate)

- vin request-urile printr-un frontend layer
- se trec printr-un load-balancing layer
- si ai servere care hosteaza copii complete ale aplicatiei
- fiecare copie contine toate feature-urile si functiile pe care le poate oferi aplicatia, fie ca le folosesti fie ca nu

SOA(granularitate aspra)

- dupa monolit a aparut SOA, care a impartit in servicii
- incapsulezi monolitii in servicii wrapper non standard
- integrezi codul vechi in noul sistem ce utilizeaza SOA

Microservicii(granularitate fina)

Design Patterns Microservicii

a)Decomposition Patterns

1)Decompose by Business Capability

- poti descompune dupa capabilitatile de business(ceva ce un business face ca sa produca o valoare)

2)Decompose by subdomain

- ai clase care nu-s usor de descompus si au elemente comune(Order Management, Order Taking, Order Delivery, etc)
- folosesti subdomenii si bounded contexts pentru a rezolva problema
- fiecare subdomeniu va avea un model si un scop care se va numi bounded context
- fiecare microserviciu va fi livrat cu contextul bounded

3)Strangler

- tot faci call-uri de la microserviciu la monolit si invers pentru fiecare domeniu.
- un serviciu poate fi impartit in diferite domenii si hostat ca servicii separate.
- ideea e sa o faci un domeniu la un moment dat.
- asta creaza doua aplicatii separate care lucreaza cot la cot in acelasi spatiu URI.
- noua aplicatie inlocuieste treptat vechiul monolit si la un moment dat poti inchide monolitul definitiv

b)Integration Patterns

4)API Gateway

- singurul punct de intrare pentru fiecare apel de microserviciu
- poate functiona ca un proxy si sa te directioneze la microserviciul potrivit, abstractizand detaliile producatorului
- poate grupa un request la mai multe servicii si sa trimiti un raspuns grupat
- poti avea mai multe API-uri pentru diferiti clienti
- Poti converti un protocol in alt protocol(actionezi pe post de protocol adapter)
- poate lua responsabilitatea de autorizare/autentificare de la microserviciu

5)Aggregator

- spune cum putem grupa datele de la diferite servicii si sa trimitem un raspuns final consumerului

- se poate face in 2 moduri: un microserviciu composite care face apeluri la microserviciile necesare, consolideaza

datele si le transforma inainte de a le trimite inapoi(adica orchestrarea de la SOA, eventual un adapter)

- API Gateway poate face request-uri la mai multe microservicii si sa uneasca datele inainte de a le trimite la consumer

- daca ai logica de business mergi pe prima varianta

6)Client-Side UI Composition

- cu microserviciile, UI-ul trebuie sa fie proiectat ca un schelet cu mai multe sectiuni per ecran/pagina

- fiecare sectiune face apel la un microserviciu backend ca sa-si ia datele.

- popular la Single Page Applications

c)Database Patterns

7)Database per Service

- ai o db per microserviciu

- privata si accesibila doar din microserviciu

8)Shared Database per Service

- aplicata doar la brownfield(aplicatii in care ai si monolit)

- ai 2-3 microservicii care impart o db

9)Command Query Responsibility Segregation(CQRS)

- cand fiecare e cu db lui, query-urile care necesita date de la mai multe microservicii nu pot fi facute

- solutia este sa imparti aplicatia in 2 parti: comanda si query.

- comanda se ocupa de CUD(Create Update Delete)

- partea de query face query-uri prin view-uri materializate

- folosit in combinatie cu pattern-ul de event sourcing unde creezi evenuri pt fiecare schimbare a datelor.

- view-urile materializate sunt tinute la curent prin abonarea la streamul de evenuri.
- sau faci o replica read-only a bazelor de date unite

10) Saga

- nu poti folosi tranzactii ACID daca-s in DB diferite
- un saga e un porces high-level de business care e fosrmat din mai multe sub-request-uri in care fiecare

face update intr-un singur serviciu.

- fiecare request are un request compensatoriu care e executat cand request-ul esueaza
- ai 2 moduri: coregrafie cand nu ai coordonator, fiecare serviciu produce si asculta eventurile altui serviciu si decide

daca o actiune trebuie luata sau nu

- orchestrare cand un obiect ia responsabilitatea pentru decizie si dirijeaza logica de business

11) API Composer

- cum implementezi query-uri care au nevoie de join de la mai multe microservicii
- faci un serviciu care face query la fiecare serviciu si da join la rezultate

12) Domain Event

- cum poate emite un serviciu evenimente cand isi da update
- organizeaza logic de business ca colectii care emit evenuri cand sunt create/updatate

d) Observability Patterns

13) Log Aggregation

- Ai un serviciu de logging pentru toata aplicatia

14) Performance Metrics

- un serviciu de metrica care colecteaza statistici
- ai 2 moduri: push(serviciul ofera metrice serviciului de metrica) si pull(serviciul de metrica ia metrice de la serviciu)

15) Distributed Tracing

- cum urmaresti un request prin toate microserviciile apelate ca sa-l rezolvi?
- un serviciu
 - asigneaza la fiecare request extern un id unic
 - da id-ul la toate serviciile
 - include id-ul in toate log-urile
 - inregistreaza informatia(start/stop time) despre request-uri si operatiile efectuate

16)Health Check

- cand un serviciu ruleaza dar nu e capabil sa se ocupe de tranzactie
- fiecare serviciu trb sa aiba un endpoint care poate fi folosit ca sa vezi sanatatea aplicatiei
- e pentru load-balance

e)Cross-Cutting Concern Patterns

17)External Configuration

- cum eviti schimbarile in cod din cauza schimbarilor de configuratie?
- externalizezi toate configuratiile cu tot cu URL-urile endpoint si credentialele. Aplicatia ar trebui sa le incarce la start
 - sau in timpul executiei
- le poti folosi apoi ca proprietati de environment de exemplu

18)Service Discovery Pattern

- creezi un serviciu registru care sa mentina metadatele fiecarui serviciu producer.
- o instanta a serviciului trebuie inregistrata in registru cand e pornita si sa se delogheze cand e oprita.
- consumerul face query la registru ca sa afle locatia serviciului.
- ai client-side si server-side discovery

19)Circuit Breaker

- cand apelezi servicii din alte servicii e posibil ca unul sa fie down si sa ai erori in cascada. Dupa un anumit numar de erori,

serviciul va da pentru o perioada prestabilita fail instant

- dupa ce expira time-out-ul, intri in perioada de test, daca dupa un nr de request-uri totul e bine se revine la normal, daca nu

cade iar tot pt timeout

20)Blue-Green Deployment

- rulezi doua instante identice de medii de productie, blue si green. Green e instanta live iar blue e noua versiune a aplicatiei

- in orice moment doar un mediu e live si serveste toate cererile.

f)Altele

21)Bulkhead

-

#####

SOLID la microservicii

S:

- un microserviciu ar trebui sa implementeze doar o functie de business
- poti sparge microserviciul coarse-grained in microservicii fine si sa le bagi un orchestrator

O:

- un microserviciu nu ar trebui sa fie niciodata modificat ca sa ofere functionalitati ocazionale sau de exceptie

- in loc sa ai un serviciu ce include cazurile exceptionale, faci un serviciu pentru ele si apelezi serviciul initial

L:

- o versiune noua a unui microserviciu ar trebui intotdeauna sa poata inlocui o versiune veche fara sa strice nimic

I:

- un microserviciu n-ar trebui sa expuna metode care nu-s direct asociate

- daca folosesti doar 30% din microserviciu, atunci ala nu e microserviciu

D:

- un microserviciu n-ar trebui sa apeleze direct alt microserviciu
- ar trebui sa foloseasca un Service Discovery (gen Docker) sau sa lase aplicatia sa determine la runtime ce ce microserviciu sa invoce

Principii de scalare a microserviciilor

Horizontal Decomposition(X axis)

- scalarea prin adaugarea mai multor instante ale microserviciului in spatele unui load balancer
- adica adaugi mai multe masini

Functional Decomposition(Vertical, Y Axis)

- imparte aplicatia in mai multe servicii diferite

Data Partitioning(Z Axis)

- rulezi mai multe copii identice, dar fiecare se ocupa doar de un subset de date
- poti sa dirijezi spre fiecare serviciu in functie de niste criterii stabilite\

Poti avea multe dependente intre microservicii si pentru o singura cerere sa pierzi mult timp

O solutie ar fi sa implementezi mecanisme de caching:

- vin cererile
- modulul lb(load balancing) transfera cererea
- serviciul verifica daca are in cache cererea respectiva
- daca da, o returneaza, daca nu, face calculele necesare

Exista:

- cache incapsulat(e bagat in serviciu)
- cache comun(e partajat intre mai multe instante ale microserviciului)

- atasament(sidecar) ai un serviciu de cache cu care comunica microserviciul(asigura cuplare slaba si elimina redundantele)
- inverse proxy in care atasezi cache-ul la lbs

Descoperire la client: te uiti intr-un service registry primesti o adresa si te conectezi la microserviciu

Descoperire la server: te conectezi la un server lb/API Gateway care se uita in service registry si-ti da forward.

Poti seta microserviciile sa se autoinregistreze sau sa se delogheze automat sau sa folosesti tool-uri externe

Modele de migrare al aplicatiilor in cloud:

- rehost, iei de la tine si pui in cloud
- refactor, modifichi aplicatia ca sa se potriveasca mai bine pe cloud
- replatform, nu faci mari modificari dar profiti de avantajele oferite
- rebuild, scrii aplicatia de la 0
- replace, retragi aplicatia si o inlocuiesti cu o noua aplicatie cloud nativa

#####

Curs 10

Reactive programming: paradigma declarativa care se ocupa de fluxuri dee date si cu propagarea schimbarilor.

Eveniment:

- e atomic
- inrudire
- comportament

Partea de comportament e cea mai importanta in proiectare si utilizare

Abordari arhitecturale

- Event-command pattern: serviciul stie endpoint-urile si API-urile celorlalte servicii si le apeleaza

- Event-first pattern: fiecare serviciu asculta pentru evenimente si emite zero sau mai multe eventuri dupa ce reactioneaza la

ce a primit

Caracteristici EDA(Event Driven Arhitecture)

- Comunicare broadcast
- reactie eficienta
- evenimente cu granularitate mica
- ontologie
- procesare evenimente complexe

#####

Cele 4 planuri ale unei arhitecturi bazate pe fluxuri de evenimente(in paranteza exemplu pentru o plata)

- 1) business function(procesarea platii)
- 2) instrumentation plane(cati bani sunt transferati, cat de repede sunt procesati)
- 3) control plane(pattern-uri de start, stop, pauza, coordonare si autoscale)
- 4) operational plane(erori, urmarire si tratare, logging)

1)Business function: functionalitatea de baza care e construita. De obicei se ocupa cu procesarea datelor.

2)Instrumentation plane: captureaza metrice care dovedesc ca functia de business se comporta acceptabil. Pot fi folosite sa emiti alerte sau sa ajuti la automatizarea infrastructurii.

3)Control plane: cand ai nevoie sa opresti tot sistemul din procesare ca sa bagi un upgrade, si nu vrei sa se opreasca toate procesarile in curs

4)Operational plane: contine partea de automatizare, logica aplicatiei, jurnale de erori, jurnale de securitate, informatii pentru restaurare, cozile

de mesaje pierdute

Serverless computing e un model de executie in cloud in care un provider ruleaza serverul si manageriaza dinamic alocarea resurselor masinii. Pretul e bazat pe cate resurse consuma

aplicatia, fata de un pret stabilit dinainte.

If your PaaS can efficiently start instances in 20ms that run for half a second, then call it serverless"

Avantaje:

- incarcari mici
- decuplare maxima
- scalabilitate
- eficiente economic

Limitari:

- timp
- memorie
- tehnologie proprietara
- solutii publice
- securitate